Helix OTA

Helix OTA — Windows OTA Support Research & Planning

> **Document ID:** `HELOTA-WIN-001` **Version:** 1.2.0-draft
> **Status:** Active **Last Updated:** 2026-03-05
> **Constitution Reference:** HelixConstitution v1 §1–§4, §7.2, §11.4.109 **Parent Roadmap:** VERSION_ROADMAP §8 (1.2.0-windows-support)

Table of Contents

---

1. Windows Update Architecture

1.1 Windows Update Ecosystem Overview

The Windows update ecosystem is a layered architecture that has evolved over two decades from a simple consumer patch-delivery mechanism into a sophisticated enterprise update management platform. Unlike Linux's fragmented distribution-specific update systems or Android's monolithic A/B partition approach, Windows provides a unified update

pipeline that can be tapped at multiple levels — from the consumer-facing Windows Update service to the enterprise-grade WSUS and Intune management planes.

For Helix OTA, the critical insight is that we are not replacing Windows Update — we are augmenting it. Our OTA client runs as a Windows Service that orchestrates update delivery for third-party applications and custom firmware deployed on Windows devices. The Windows Update infrastructure provides the delivery transport and scheduling primitives; Helix OTA provides the fleet management, rollout control, and telemetry that Windows Update alone does not offer.

The Windows update stack consists of five principal layers:

1. **Windows Update Agent (WUA):** The local client-side engine that scans for, downloads, and installs updates.
2. **Windows Server Update Services (WSUS):** An on-premises server role that acts as an upstream proxy and approval gate between Microsoft Update and enterprise clients.
3. **Windows Update for Business (WUfB):** A cloud-based policy service that enables deferral rings, feature update pausing, and quality update controls without on-premises infrastructure.
4. **Intune / MECM:** Full device management platforms (cloud and on-premises respectively) that layer application deployment, compliance policies, and conditional access on top of WUfB.
5. **Third-party update catalogs:** A mechanism by which ISVs can publish update metadata to WSUS or Intune, enabling distribution through the standard Windows Update pipeline.

1.2 Windows Update Agent (WUA) API

The Windows Update Agent is the local service (`wuauserv`) responsible for scanning, downloading, and installing updates on every Windows machine. It exposes a COM-based API that can be consumed from C++, C#, VBScript, and PowerShell. The WUA API is the lowest-level programmatic interface for controlling Windows Update behavior.

Key COM interfaces:

Interface	Purpose
`IUpdateSession`	Entry point for creating update search,

Interface	Purpose
	download, and installation operations
`IUpdateSearcher`	Searches for updates matching specified criteria
`IUpdateDownloader`	Downloads update content for a set of updates
`IUpdateInstaller`	Installs downloaded updates
`IAutomaticUpdates`	Controls the Automatic Updates configuration
`IUpdateServiceManager`	Manages update service registrations (Microsoft Update, WSUS, custom)

PowerShell interaction with WUA:

```
# Create an update session and search for available updates
$Session = New-Object -ComObject Microsoft.Update.Session
$Searcher = $Session.CreateUpdateSearcher()

# Search for software updates that are not yet installed
$Result = $Searcher.Search("IsInstalled=0 and
        Type='Software'")

Write-Host "Found $($Result.Updates.Count) available
        updates"

foreach ($Update in $Result.Updates) {
    Write-Host "  KB Article: $($Update.KBArticleIDs -join
        ', ')"
    Write-Host "  Title: $($Update.Title)"
    Write-Host "  Description: $($Update.Description)"
    Write-Host "  Size: $
        ([math]::Round($Update.MaxDownloadSize / 1MB, 2))
        MB"
    Write-Host ""
}

# Download and install available updates
if ($Result.Updates.Count -gt 0) {
    $Downloader = $Session.CreateUpdateDownloader()
    $Downloader.Updates = $Result.Updates
    $DownloadResult = $Downloader.Download()

    Write-Host "Download result code: $
        ($DownloadResult.ResultCode)"

    if ($DownloadResult.ResultCode -eq 2) { # Succeeded
```

```
        $Installer = $Session.CreateUpdateInstaller()
        $Installer.Updates = $Result.Updates
        $InstallResult = $Installer.Install()

        Write-Host "Install result code: $
        ($InstallResult.ResultCode)"
        Write-Host "Reboot required: $
        ($InstallResult.RebootRequired)"
    }
}
```

WUA result codes:

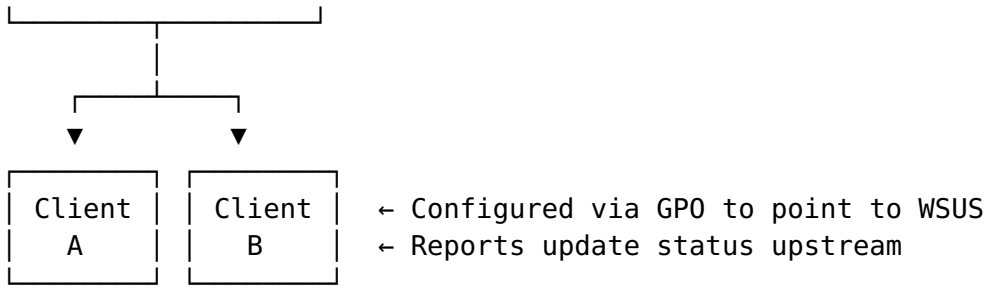
Code	Value	Meaning
`orcNotStarted`	0	Operation not started
`orcInProgress`	1	Operation in progress
`orcSucceeded`	2	Operation completed successfully
`orcSucceededWithErrors`	3	Operation succeeded with errors
`orcFailed`	4	Operation failed
`orcAborted`	5	Operation aborted

For Helix OTA, the WUA API is relevant primarily for querying the Windows Update state — detecting pending reboots, determining the current OS build, and checking whether security updates are up to date. We do not use WUA to install our own updates because WUA requires updates to be packaged in Microsoft's cabinet (.cab) format with signed metadata — a process that requires a Microsoft Partnership and a Publisher Certificate. Instead, we use WUA as an observability layer and rely on our own delivery mechanism for Helix-managed payloads.

1.3 WSUS (Windows Server Update Services)

WSUS is a server role that enables administrators to manage the distribution of updates through a corporate network. It acts as an internal mirror of Microsoft Update, allowing administrators to approve or decline updates before they reach client machines.

Architecture:



WSUS relevance for Helix OTA: WSUS supports third-party update catalogs. An ISV can create a custom update catalog (an XML file conforming to the WSUS catalog schema) and import it into WSUS. This allows enterprise administrators to distribute third-party updates through the same WSUS pipeline they use for Microsoft updates.

Third-party catalog format:

```
<?xml version="1.0" encoding="utf-8"?>
<catalog xmlns="http://schemas.microsoft.com/msus/2005/12/
        UpdateCatalog">
  <Properties>
    <PublisherName>Helix OTA Systems</PublisherName>
    <PublisherId>helix-ota-systems</PublisherId>
  </Properties>
  <Updates>
    <Update>
      <ID>helix-ota-agent-1.2.0</ID>
      <Title>Helix OTA Agent 1.2.0</Title>
      <Description>Over-the-air update agent for Windows
        devices</Description>
      <Classification>Applications</Classification>
      <Bundled>false</Bundled>
      <Payload>
        <DownloadLocation>https://ota.helix.dev/updates/
        agent-1.2.0.msi</DownloadLocation>
        <FileSize>12582912</FileSize>
        <DigestAlgorithm>SHA256</DigestAlgorithm>
        <Digest>4a2f8c1e9d3b7a6f0e5c8d2b1a4f7e3c6d9b2e5a8f1c4d7e0b3a6f9c2e5d8b1
        Digest>
      </Payload>
      <InstallCommand>msiexec /i agent-1.2.0.msi /qn
        REBOOT=ReallySuppress</InstallCommand>
      <UninstallCommand>msiexec /x {HELIX-OTA-AGENT-GUID} /
        qn</UninstallCommand>
    </Update>
  </Updates>
</catalog>
```

However, the WSUS third-party catalog approach has significant limitations: it requires enterprise customers to have WSUS deployed, it does not support consumer devices, and the catalog import process is manual and

error-prone. For Helix OTA, we support WSUS catalog import as an optional integration point for enterprise customers, but our primary distribution mechanism is direct delivery via the Helix OTA server.

1.4 Windows Update for Business (WUfB)

Windows Update for Business is a cloud-based policy service that enables deferral rings, feature update pausing, and quality update controls without on-premises WSUS infrastructure. WUfB is configured via Group Policy, MDM (Mobile Device Management) policies, or Intune.

Deferral ring model:

WUfB uses a ring-based deployment model that maps directly to Helix OTA's rollout group concept:

Ring	Deferral	Purpose	Helix Equivalent
Preview	0 days	Early testing	`canary`
Early	7 days	Limited rollout	`early_adopter`
Broad	14 days	General availability	`stable`
Late	30 days	Conservative	`conservative`

MDM policy configuration (via Intune OMA-URI):

```
<!-- Defer quality updates by 7 days -->
<Replace>
  <CmdID>1</CmdID>
  <Item>
    <Target>
      <LocURI>./Vendor/MSFT/Policy/Config/Update/
        DeferQualityUpdatesPeriodInDays</LocURI>
    </Target>
    <Meta>
      <Format>int</Format>
    </Meta>
    <Data>7</Data>
  </Item>
</Replace>

<!-- Pause feature updates until a specific date -->
<Replace>
  <CmdID>2</CmdID>
  <Item>
    <Target>
      <LocURI>./Vendor/MSFT/Policy/Config/Update/
        PauseFeatureUpdatesStartTime</LocURI>
```

```
    </Target>
    <Meta>
      <Format>chr</Format>
    </Meta>
    <Data>2026-04-01</Data>
  </Item>
</Replace>
```

WUfB relevance for Helix OTA: We integrate with WUfB at the policy level. When a Helix OTA administrator pauses a rollout, we can optionally signal WUfB to pause Windows Update on the affected devices as well, ensuring that OS-level and application-level updates are coordinated. This prevents scenarios where a Windows feature update changes a system dependency that breaks a Helix-managed application.

1.5 Intune / MECM Integration

Microsoft Intune (cloud) and Microsoft Endpoint Configuration Manager (MECM, on-premises) provide full device management capabilities including application deployment, compliance policies, and conditional access.

Intune Win32 app deployment:

Intune can deploy Win32 applications packaged in the `.intunewin` format. This is the most direct integration point for Helix OTA — we can package our OTA client as an Intune Win32 app and let enterprise customers deploy it through their existing Intune infrastructure.

```
# Package a Helix OTA MSI for Intune deployment using the
# Microsoft Win32 Content Prep Tool
IntuneWinAppUtil.exe `
    -c "C:\Build\helix-ota-agent" `
    -s "helix-ota-agent-1.2.0.msi" `
    -o "C:\Build\output" `
    -q

# The resulting .intunewin file can be uploaded to Intune
# with the following install/uninstall commands:
# Install:   msiexec /i helix-ota-agent-1.2.0.msi /qn
# Uninstall: msiexec /x {HELIX-OTA-AGENT-GUID} /qn
```

MECM application deployment:

For on-premises environments, MECM (formerly SCCM) can deploy applications using packages, programs, or the newer application model. The Helix OTA MSI can be imported directly into MECM as an application with detection logic based on registry keys or file versions.

```
# MECM detection logic (PowerShell script)
$RegPath = "HKLM:\SOFTWARE\Helix OTA\Agent"
$ExpectedVersion = "1.2.0"

try {
    $InstalledVersion = (Get-ItemProperty -Path $RegPath
         -Name "Version" -ErrorAction Stop).Version
    if ($InstalledVersion -eq $ExpectedVersion) {
        Write-Host "Detected"
        exit 0
    }
} catch {}

Write-Host "Not Detected"
exit 1
```

Integration strategy: Helix OTA provides a first-party client that manages its own updates independently. However, for enterprise customers who require all software to be deployed through Intune or MECM, we provide:

1. `.intunewin` packages for Intune deployment
2. MECM application definitions with detection logic
3. Compliance policy templates that verify the Helix OTA agent is running and up to date
4. A PowerShell module (`HelixOta`) that enterprise administrators can use to query and control the Helix OTA agent via Intune's remote PowerShell capability

1.6 Third-Party Update Mechanisms

Beyond the Microsoft-provided update infrastructure, several third-party update mechanisms are relevant to the Windows ecosystem:

Winget (Windows Package Manager): Microsoft's official package manager for Windows, included in Windows 10 1709+ and Windows 11. Winget uses a community-maintained manifest repository (winget-pkgs on GitHub) and supports MSIX, MSI, and EXE installers. Winget is the most promising distribution channel for Helix OTA because it is built into Windows, does not require enterprise infrastructure, and supports silent installation.

Chocolatey: A popular third-party package manager with a large community repository. Chocolatey uses NuGet packages and PowerShell for installation. It is widely used in enterprise environments but requires separate licensing for commercial use of its Pro/Business features.

Scoop: A command-line installer for Windows that focuses on portable, non-administrative installations. Scoop is popular with developers but less suited for system-level OTA agents that require administrator privileges.

Our approach: We prioritize Winget as the primary distribution channel (see §3.3), with MSI as the fallback for environments where Winget is not available or not desired. Chocolatey support is provided as a community contribution. Scoop is not supported because the Helix OTA agent requires system-level installation (Windows Service) that is incompatible with Scoop's user-space model.

---

2. Windows Service Client Design

2.1 Windows Service Architecture for OTA Client

The Helix OTA client on Windows runs as a Windows Service. This is non-negotiable — a Windows Service is the only mechanism that provides:

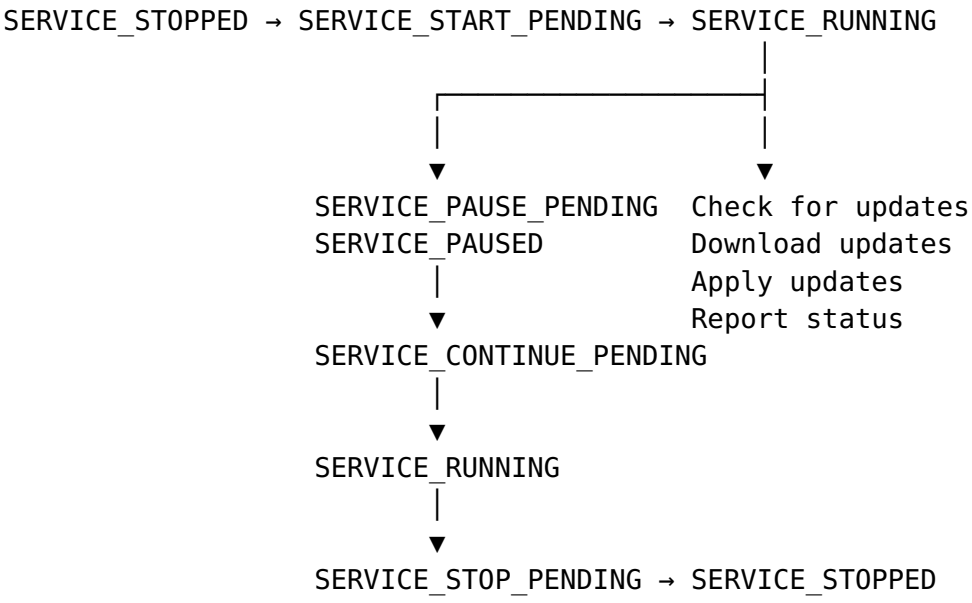
1. **Start before user logon:** The OTA client must be able to check for and apply updates before any user logs in, ensuring that devices are up to date when the user first interacts with them.
2. **Run in Session 0:** Windows Services run in Session 0, which is isolated from user sessions. This prevents the OTA client from interfering with the user's desktop and prevents the user from accidentally terminating the OTA client.
3. **Automatic recovery:** The Windows Service Control Manager (SCM) can automatically restart a crashed service with configurable failure actions.
4. **Privilege isolation:** The service runs under a dedicated service account (or LocalSystem/NetworkService) with precisely defined privileges.

Service configuration:

```
ServiceName: HelixOtaAgent
DisplayName: "Helix OTA Update Agent"
```

```
Description:
        "Manages over-the-air updates for Helix-managed
        software on this device"
StartType: auto  # Automatic start on boot
ServiceSidType: unrestricted  # Required for network access
DelayedAutoStart: true  # Start after other auto-start
        services for faster boot
FailureActions:
  - Type: restart
    Delay: 5000  # 5 seconds
  - Type: restart
    Delay: 30000  # 30 seconds
  - Type: restart
    Delay: 60000  # 60 seconds
  - Type: none  # Give up after 3 failures
    Delay: 0
ResetFailureCount: 86400  # Reset failure counter after 24
        hours
```

Service lifecycle:



2.2 Go Implementation as a Windows Service

Go has excellent Windows Service support through the `golang.org/x/sys/windows/svc` package and the third-party `golang.org/x/sys/windows/svc/mgr` package. The Helix OTA agent is compiled as a Windows service executable.

```
package main

import (
    "context"
    "fmt"
    "log"
    "os"
```

```
    "path/filepath"
    "time"

    "golang.org/x/sys/windows/svc"
    "golang.org/x/sys/windows/svc/mgr"
    "golang.org/x/sys/windows/registry"
)

const serviceName = "HelixOtaAgent"

// otaService implements svc.Handler for the Windows Service
        Control Manager.
type otaService struct {
    agent    *OTAAgent
    stopChan chan struct{}
}

// Execute is the main service entry point called by the
        Windows SCM.
// It must conform to the svc.Handler interface.
func (s *otaService) Execute(args []string, r <-chan
        svc.ChangeRequest, status chan<- svc.Status)
        (bool, uint32) {
    // Report starting status
    status <- svc.Status{State: svc.StartPending}

    // Initialize the OTA agent
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    var err error
    s.agent, err = NewOTAAgent(ctx)
    if err != nil {
        log.Printf("FATAL: Failed to initialize OTA agent:
        %v", err)
        return true, 1 // Service failed
    }

    // Start the agent's main loop in a goroutine
    agentDone := make(chan error, 1)
    go func() {
        agentDone <- s.agent.Run(ctx)
    }()

    // Report running status
    status <- svc.Status{
        State:   svc.Running,
        Accepts: svc.AcceptStop | svc.AcceptShutdown |
        svc.AcceptPauseAndContinue,
    }

    // Main service loop: handle SCM control requests
    for {
```

```
        select {
        case c := <-r:
            switch c.Cmd {
            case svc.Interrogate:
                status <- c.CurrentStatus

            case svc.Stop, svc.Shutdown:
                status <- svc.Status{State: svc.StopPending}
                cancel() // Signal the agent to stop
                <-agentDone
                return false, 0 // Clean shutdown

            case svc.Pause:
                status <- svc.Status{State:
        svc.PausePending}
                s.agent.Pause()
                status <- svc.Status{
                    State:   svc.Paused,
                    Accepts: svc.AcceptStop |
        svc.AcceptShutdown | svc.AcceptPauseAndContinue,
                }

            case svc.Continue:
                status <- svc.Status{State:
        svc.ContinuePending}
                s.agent.Resume()
                status <- svc.Status{
                    State:   svc.Running,
                    Accepts: svc.AcceptStop |
        svc.AcceptShutdown | svc.AcceptPauseAndContinue,
                }

            default:
                log.Printf("WARNING: Unhandled control
        request: %v", c.Cmd)
            }

        case err := <-agentDone:
            if err != nil {
                log.Printf("Agent exited with error: %v",
        err)
                return true, 1 // Service failed
            }
            return false, 0 // Agent exited cleanly
        }
    }
}

// InstallService registers the OTA agent as a Windows
        Service.
func InstallService(exePath string) error {
    m, err := mgr.Connect()
    if err != nil {
```

```
    return fmt.Errorf("failed to connect to service
    manager: %w", err)
}
defer m.Disconnect()

// Check if the service already exists
s, err := m.OpenService(serviceName)
if err == nil {
    s.Close()
    return fmt.Errorf("service %s already exists",
    serviceName)
}

// Create the service
s, err = m.CreateService(serviceName, exePath,
    mgr.Config{
    DisplayName:      "Helix OTA Update Agent",
    Description:      "Manages over-the-air updates for
    Helix-managed software",
    StartType:        mgr.StartAutomatic,
    DelayedAutoStart: true,
    ServiceType:      mgr.Interactive,
}, "--service")
if err != nil {
    return fmt.Errorf("failed to create service: %w",
    err)
}
defer s.Close()

// Configure failure actions: restart on failure, up to
    3 times
// with increasing delays
failActions := mgr.RecoveryAction{
    Type:  mgr.ServiceRestart,
    Delay: 5 * time.Second,
}
if err := s.SetRecoveryActions(
    []mgr.RecoveryAction{
        {Type: mgr.ServiceRestart, Delay: 5 *
    time.Second},
        {Type: mgr.ServiceRestart, Delay: 30 *
    time.Second},
        {Type: mgr.ServiceRestart, Delay: 60 *
    time.Second},
    },
    86400, // Reset failure count after 24 hours
); err != nil {

    log.Printf("WARNING: Failed to set recovery
    actions: %v", err)
}

// Configure the service SID type as unrestricted for
    network access
```

```
    if err := configureServiceSidType(s, "unrestricted");
        err != nil {

        log.Printf("WARNING: Failed to configure service
        SID type: %v", err)
    }

    return nil
}

// UninstallService removes the OTA agent Windows Service.
func UninstallService() error {
    m, err := mgr.Connect()
    if err != nil {
        return fmt.Errorf("failed to connect to service
        manager: %w", err)
    }
    defer m.Disconnect()

    s, err := m.OpenService(serviceName)
    if err != nil {
        return fmt.Errorf("service %s not found: %w",
        serviceName, err)
    }
    defer s.Close()

    // Stop the service first
    status, err := s.Query()
    if err == nil && status.State != svc.Stopped {
        s.Control(svc.Stop)
        // Wait for the service to stop
        for i := 0; i < 30; i++ {
            time.Sleep(time.Second)
            status, err = s.Query()
            if err != nil || status.State == svc.Stopped {
                break
            }
        }
    }

    if err := s.Delete(); err != nil {
        return fmt.Errorf("failed to delete service: %w",
        err)
    }

    return nil
}

func main() {
    // Check if we're running as a Windows Service or
        interactively
    isService, err := svc.IsWindowsService()
    if err != nil {
```

```
        log.Fatalf("Failed to determine if running as
        service: %v", err)
    }

    if isService {
        // Run as a Windows Service
        s := &otaService{stopChan: make(chan struct{})}
        if err := svc.Run(serviceName, s); err != nil {
            log.Fatalf("Service execution failed: %v", err)
        }
    } else {
        // Running interactively – handle CLI commands
        if len(os.Args) < 2 {
            fmt.Println("Usage: helix-ota-agent [--
        service|--install|--uninstall|--version]")
            os.Exit(1)
        }
        switch os.Args[1] {
        case "--install":
            exePath, _ := filepath.Abs(os.Args[0])
            if err := InstallService(exePath); err != nil {
                log.Fatalf("Install failed: %v", err)
            }
            fmt.Println("Helix OTA Agent service installed
        successfully")
        case "--uninstall":
            if err := UninstallService(); err != nil {
                log.Fatalf("Uninstall failed: %v", err)
            }

        fmt.Println("Helix OTA Agent service uninstalled
        successfully")
        case "--version":
            fmt.Println("Helix OTA Agent v1.2.0")
        default:
            fmt.Printf("Unknown command: %s\n", os.Args[1])
            os.Exit(1)
        }
    }
}
```

2.3 Background Intelligent Transfer Service (BITS) for Downloads

BITS (Background Intelligent Transfer Service) is a Windows component that transfers files in the background using idle network bandwidth. It is the same technology that Windows Update uses to download updates. BITS is critical for OTA because it:

1. **Respects network cost:** BITS can be configured to only transfer on unmetered connections, preventing

unexpected cellular charges on Windows devices with LTE.
2. **Resumes after interruptions:** If the network drops or the system reboots, BITS automatically resumes the download from where it left off.
3. **Uses idle bandwidth:** BITS monitors network usage and only uses bandwidth that is not needed by foreground applications.
4. **Provides progress notifications:** BITS supports COM callbacks for download progress.

Go BITS integration:

```
package bits

import (
    "context"
    "fmt"
    "time"
    "unsafe"

    "golang.org/x/sys/windows"
)

// BITSJob represents a BITS transfer job.
type BITSJob struct {
    jobName    string
    remoteURL  string
    localPath  string
    jobHandle  uintptr // IBackgroundCopyJob COM interface
    manager    *BITSManager
}

// BITSManager manages BITS transfer jobs.
type BITSManager struct {
    managerHandle uintptr // IBackgroundCopyManager COM
        interface
}

// NewBITSManager creates a new BITS manager.
func NewBITSManager() (*BITSManager, error) {
    // Initialize COM
    if err := windows.CoInitializeEx(0,
        windows.COINIT_MULTITHREADED); err != nil {
        return nil, fmt.Errorf("COM initialization failed:
        %w", err)
    }

    m := &BITSManager{}
    // Create IBackgroundCopyManager instance via
        CoCreateInstance
    // CLSID_BackgroundCopyManager =
        {4991D34B-80A1-4291-83B6-3328366B9097}
```

```
    var managerHandle uintptr
    clsid := windows.GUID{0x4991D34B, 0x80A1, 0x4291,
        [8]byte{0x83, 0xB6, 0x33, 0x28, 0x36, 0x6B, 0x90,
        0x97}}
    iid := windows.GUID{0x5CE34C0D, 0x0DC9, 0x4C1F,
        [8]byte{0x89, 0x7C, 0xDA, 0xA1, 0xB7, 0x8C, 0xEE,
        0x7C}}

    hr := windows.CoCreateInstance(
        &clsid,
        0,
        windows.CLSCTX_LOCAL_SERVER,
        &iid,
        &managerHandle,
    )
    if hr != 0 {
        windows.CoUninitialize()
        return nil, fmt.Errorf("CoCreateInstance failed:
        0x%08X", hr)
    }
    m.managerHandle = managerHandle

    return m, nil
}

// CreateJob creates a new BITS download job.
func (m *BITSManager) CreateJob(ctx context.Context,
        jobName, remoteURL, localPath string) (*BITSJob,
        error) {
    job := &BITSJob{
        jobName:   jobName,
        remoteURL: remoteURL,
        localPath: localPath,
        manager:   m,
    }

    // Create the job via IBackgroundCopyManager::CreateJob
    // This is simplified — actual implementation uses COM
        vtable calls
    // In practice, use the github.com/go-ole/go-ole library
        for COM interop

    return job, nil
}

// WaitForCompletion blocks until the BITS job completes or
        the context is cancelled.
func (j *BITSJob) WaitForCompletion(ctx context.Context)
        error {
    ticker := time.NewTicker(2 * time.Second)
    defer ticker.Stop()

    for {
```

```
        select {
        case <-ctx.Done():
            // Cancel the BITS job on context cancellation
            j.Cancel()
            return ctx.Err()
        case <-ticker.C:
            state := j.GetState()
            switch state {
            case 1: // BG_JOB_STATE_QUEUED
                // Job is queued, waiting for bandwidth
            case 2: // BG_JOB_STATE_CONNECTING
                // Job is connecting to the server
            case 3: // BG_JOB_STATE_TRANSFERRING
                progress := j.GetProgress()
                log.Printf("BITS download progress: %d/%d
        bytes (%.1f%%)",
                    progress.BytesTransferred,
                    progress.BytesTotal,
                    float64(progress.BytesTransferred)/
        float64(progress.BytesTotal)*100,
                )
            case 4: // BG_JOB_STATE_TRANSFERRED
                j.Complete()
                return nil
            case 5: // BG_JOB_STATE_ACKNOWLEDGED
                return nil
            case 6: // BG_JOB_STATE_CANCELLED
                return fmt.Errorf("BITS job was cancelled")
            case 7: // BG_JOB_STATE_ERROR
                errDetail := j.GetError()
                return fmt.Errorf("BITS job failed: %s",
        errDetail)
            }
        }
    }
}

// GetState returns the current BITS job state.
func (j *BITSJob) GetState() uint32 {
    // Call IBackgroundCopyJob::GetState via COM vtable
    // Simplified — actual implementation uses go-ole
    return 0
}

// GetProgress returns the current download progress.
func (j *BITSJob) GetProgress() BITSSProgress {
    // Call IBackgroundCopyJob::GetProgress via COM vtable
    return BITSSProgress{}
}

// Complete finalizes a successfully transferred BITS job.
func (j *BITSJob) Complete() error {
```

```
    // Call IBackgroundCopyJob::Complete via COM vtable
    return nil
}

// Cancel cancels a BITS job.
func (j *BITSJob) Cancel() error {
    // Call IBackgroundCopyJob::Cancel via COM vtable
    return nil
}

// BITSSProgress represents BITS download progress.
type BITSSProgress struct {
    BytesTotal       uint64
    BytesTransferred uint64
    FilesTotal       uint32
    FilesTransferred uint32
}

// Close releases BITS resources.
func (m *BITSManager) Close() {
    if m.managerHandle != 0 {
        // Release COM interface
        windows.CoTaskMemFree(unsafe.Pointer(m.managerHandle))
        m.managerHandle = 0
    }
    windows.CoUninitialize()
}
```

BITS priority model:

Priority	Value	Use Case
`BG_JOB_PRIORITY_FOREGROUND`	0	Interactive, user-initiated downloads
`BG_JOB_PRIORITY_HIGH`	1	Important updates that should download quickly
`BG_JOB_PRIORITY_NORMAL`	2	Standard update downloads (default for Helix OTA)
`BG_JOB_PRIORITY_LOW`	3	Background pre-fetching of future updates

2.4 Windows Event Log Integration

The Helix OTA agent logs all significant events to the Windows Event Log. This ensures that update operations are visible to system administrators through the standard Windows event viewing tools (Event Viewer, `wevtutil`, PowerShell `Get-WinEvent`).

```
package eventlog

import (
    "fmt"
    "golang.org/x/sys/windows"
    "golang.org/x/sys/windows/registry"
)

const (
    eventSource = "HelixOtaAgent"
    eventLogKey =
        `SYSTEM\CurrentControlSet\Services\EventLog\Application\HelixOtaAgent`
)

// Event IDs for structured event logging
const (
    EventIDServiceStart      = 1001
    EventIDServiceStop       = 1002
    EventIDUpdateCheckStart  = 2001
    EventIDUpdateAvailable   = 2002
    EventIDUpdateNotFound    = 2003
    EventIDDownloadStart     = 3001
    EventIDDownloadProgress  = 3002
    EventIDDownloadComplete  = 3003
    EventIDDownloadFailed    = 3004
    EventIDInstallStart      = 4001
    EventIDInstallComplete   = 4002
    EventIDInstallFailed     = 4003
    EventIDRollbackStart     = 5001
    EventIDRollbackComplete  = 5002
    EventIDRollbackFailed    = 5003
)

// RegisterEventSource registers the Helix OTA agent as an
        Event Log source.
// This must be called during installation (requires
        administrator privileges).
func RegisterEventSource(exePath string) error {
    k, _, err := registry.CreateKey(registry.LOCAL_MACHINE,
        eventLogKey, registry.ALL_ACCESS)
    if err != nil {
        return fmt.Errorf("failed to create event log
        registry key: %w", err)
    }
    defer k.Close()
```

```
    if err := k.SetStringValue("EventMessageFile",
        exePath); err != nil {
        return fmt.Errorf("failed to set EventMessageFile:
        %w", err)
    }
    if err := k.SetDWordValue("TypesSupported", 0x07); err !
        = nil { // Error | Warning | Information
        return fmt.Errorf("failed to set TypesSupported:
        %w", err)
    }

    return nil
}

// Logger wraps the Windows Event Log for structured
        logging.
type Logger struct {
    handle windows.Handle
}

// NewLogger opens the event log for writing.
func NewLogger() (*Logger, error) {
    handle, err := windows.RegisterEventSource(nil,
        windows.StringToUTF16Ptr(eventSource))
    if err != nil {
        return nil, fmt.Errorf("failed to register event
        source: %w", err)
    }
    return &Logger{handle: handle}, nil
}

// Info logs an informational event.
func (l *Logger) Info(eventID uint32, format string,
        args ...interface{}) {
    l.report(windows.EVENTLOG_INFORMATION_TYPE, eventID,
        format, args...)
}

// Warning logs a warning event.
func (l *Logger) Warning(eventID uint32, format string,
        args ...interface{}) {
    l.report(windows.EVENTLOG_WARNING_TYPE, eventID,
        format, args...)
}

// Error logs an error event.
func (l *Logger) Error(eventID uint32, format string,
        args ...interface{}) {
    l.report(windows.EVENTLOG_ERROR_TYPE, eventID, format,
        args...)
}
```

```
func (l *Logger) report(eventType uint16, eventID uint32,
        format string, args ...interface{}) {
    msg := fmt.Sprintf(format, args...)
    msgPtr := windows.StringToUTF16Ptr(msg)
    windows.ReportEvent(l.handle, eventType, 0, eventID, 0,
        1, 0, &msgPtr, nil)
}

// Close releases the event log handle.
func (l *Logger) Close() error {
    return windows.DeregisterEventSource(l.handle)
}
```

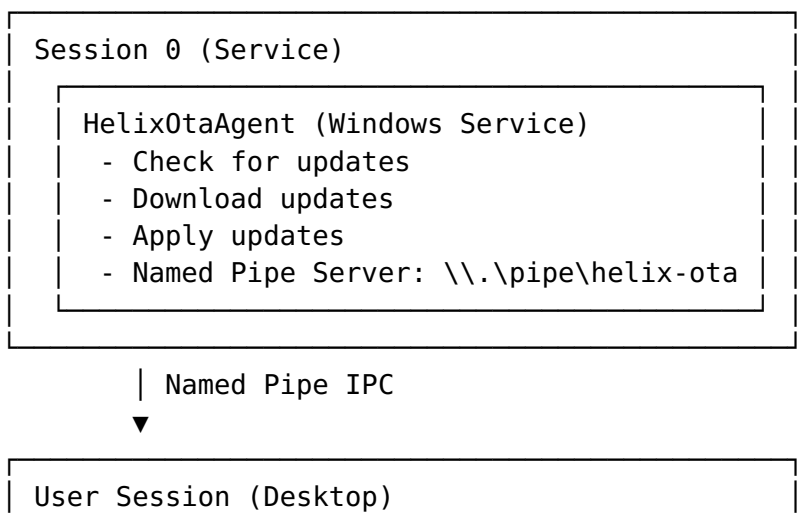
PowerShell event log query:

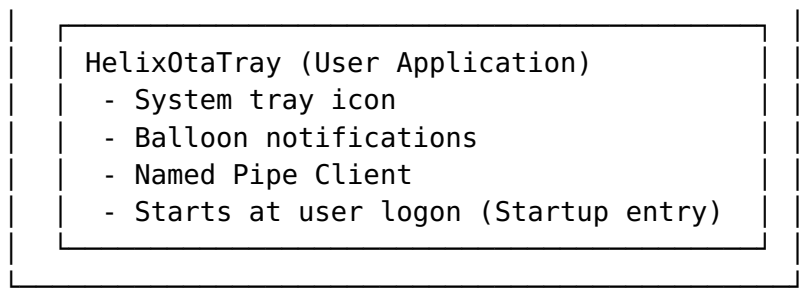
```
# Query Helix OTA events from the Windows Event Log
Get-WinEvent -LogName Application -FilterXPath
        "*[Provider[@Name='HelixOtaAgent']]" |
    Select-Object TimeCreated, Id, LevelDisplayName,
        Message |
    Format-Table -AutoSize

# Query only error events from the last 24 hours
$StartTime = (Get-Date).AddHours(-24)
Get-WinEvent -LogName Application -FilterXPath
        "*[Provider[@Name='HelixOtaAgent'] and Level=2 and
        TimeCreated >= '$StartTime']" |
    Select-Object TimeCreated, Id, Message
```

2.5 System Tray Notification

While the OTA agent runs as a Windows Service in Session 0, it must communicate update status to the user's desktop session. This requires an IPC (Inter-Process Communication) bridge between the service and a user-space tray application.

Architecture:





Named pipe protocol for service-to-tray communication:

```go
package ipc

import (
    "encoding/json"
    "fmt"
    "net"
    "sync"
)

// MessageType defines the IPC message types.
type MessageType string

const (
    MsgUpdateAvailable MessageType = "update_available"
    MsgDownloadProgress MessageType = "download_progress"
    MsgUpdateReady     MessageType = "update_ready"
    MsgUpdateApplied   MessageType = "update_applied"
    MsgUpdateFailed    MessageType = "update_failed"
    MsgRebootRequired  MessageType = "reboot_required"
    MsgStatusQuery     MessageType = "status_query"
    MsgStatusResponse  MessageType = "status_response"
)

// IPCMessage is the wire format for service-to-tray
        communication.
type IPCMessage struct {
    Type      MessageType            `json:"type"`
    Timestamp string                 `json:"timestamp"`
    Payload   map[string]interface{} `json:"payload"`
}

// PipeServer listens on a named pipe for tray application
        connections.
type PipeServer struct {
    pipePath string
    listener net.Listener
    mu       sync.Mutex
    clients  map[net.Conn]struct{}
}
```

```
// NewPipeServer creates a new named pipe server.
func NewPipeServer() *PipeServer {
    return &PipeServer{
        pipePath: `\\.\pipe\helix-ota`,
        clients:  make(map[net.Conn]struct{}),
    }
}

// Start begins listening for tray application connections.
func (s *PipeServer) Start() error {
    var err error
    s.listener, err = ListenPipe(s.pipePath)
    if err != nil {
        return fmt.Errorf("failed to listen on pipe: %w",
        err)
    }

    go s.acceptLoop()
    return nil
}

// Broadcast sends a message to all connected tray
        applications.
func (s *PipeServer) Broadcast(msg IPCMessage) error {
    data, err := json.Marshal(msg)
    if err != nil {
        return fmt.Errorf("failed to marshal message: %w",
        err)
    }
    data = append(data, '\n')

    s.mu.Lock()
    defer s.mu.Unlock()

    for conn := range s.clients {
        if _, err := conn.Write(data); err != nil {
            delete(s.clients, conn)
            conn.Close()
        }
    }
    return nil
}

func (s *PipeServer) acceptLoop() {
    for {
        conn, err := s.listener.Accept()
        if err != nil {
            return
        }
        s.mu.Lock()
        s.clients[conn] = struct{}{}
        s.mu.Unlock()
```

```
    }
}
```

2.6 Auto-Start and Recovery Configuration

The Helix OTA agent must start automatically on boot and recover from failures. Beyond the Windows Service recovery actions configured in §2.2, we implement additional resilience:

Watchdog timer: The agent exposes a heartbeat through the Windows Service checkpoint mechanism. If the agent's main loop stalls (no heartbeat for 5 minutes), the SCM treats this as a service failure and triggers the configured recovery action.

Health check script: A secondary PowerShell script runs as a Scheduled Task every 15 minutes to verify the service is responsive:

```
# HelixOtaHealthCheck.ps1 — Scheduled task for service
        health monitoring
$ServiceName = "HelixOtaAgent"
$MaxStallMinutes = 5

$Service = Get-Service -Name $ServiceName -ErrorAction
        SilentlyContinue

if (-not $Service) {
    Write-Error "Service $ServiceName not found"
    exit 1
}

if ($Service.Status -ne 'Running') {
    Write-Warning "Service $ServiceName is not running
        (status: $($Service.Status))"
    # Attempt to start the service
    Start-Service -Name $ServiceName -ErrorAction
        SilentlyContinue
    Start-Sleep -Seconds 10
    $Service.Refresh()
    if ($Service.Status -ne 'Running') {
        Write-Error "Failed to start service $ServiceName"
        # Log to Windows Event Log
        Write-EventLog -LogName Application -Source
        "HelixOtaWatchdog" `
            -EntryType Error -EventId 9001 `
            -Message "Helix OTA Agent service failed to
        start after health check intervention"
        exit 1
    }
}
```

```powershell
# Check if the service is responsive via named pipe
try {
    $Pipe = New-Object
        System.IO.Pipes.NamedPipeClientStream(
        ".", "helix-ota",
        [System.IO.Pipes.PipeDirection]::InOut, 5000
    )
    $Pipe.Connect(5000)
    $Reader = New-Object System.IO.StreamReader($Pipe)
    $Writer = New-Object System.IO.StreamWriter($Pipe)

    $Query = '{"type":"status_query","timestamp":"' + (Get-
        Date -Format "o") + '"}'
    $Writer.WriteLine($Query)
    $Writer.Flush()

    $Pipe.Close()
    Write-Host "Service health check passed"
    exit 0
} catch {
    Write-Warning
        "Service is running but not responsive via IPC: $_"
    # Restart the service
    Restart-Service -Name $ServiceName -Force
    exit 0
}
```

---

3. MSI/MSIX Package Distribution

3.1 MSI Package Creation and Management

Windows Installer (MSI) is the standard installation technology for Windows desktop applications. MSI packages provide transactional installation (automatic rollback on failure), standardized uninstall, and enterprise deployment support.

Helix OTA MSI structure:

```
helix-ota-agent-1.2.0.msi
├── File table
│   ├── helix-ota-agent.exe       (Service executable)
│   ├── helix-ota-tray.exe        (Tray application)
│   ├── helix-ota-cli.exe         (Command-line interface)
│   ├── config.yaml               (Default configuration)
│   └── uninstall.cmd             (Cleanup script)
├── Registry table
│   ├── HKLM\SOFTWARE\Helix OTA\Agent\Version = "1.2.0"
│   ├── HKLM\SOFTWARE\Helix OTA\Agent\InstallPath =
"[INSTALLDIR]"
```

```
│   └──
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\Hel
├── Service table
│   └── HelixOtaAgent (auto-start, LocalSystem)
├── Custom actions
│   ├── ConfigureEventLog (deferred, after InstallFiles)
│   ├── RegisterEventSource (deferred, after InstallFiles)
│   └── StartService (commit, after InstallFinalize)
└── Properties
    ├── Manufacturer = "Helix OTA Systems"
    ├── ARPSYSTEMCOMPONENT = 0 (visible in Add/Remove
Programs)
    ├── MSIINSTALLPERUSER = "" (machine-wide installation)
    └── ALLUSERS = 1
```

MSI generation with Go:

```go
package msigen

import (
    "embed"
    "fmt"
    "os/exec"
    "path/filepath"
    "text/template"
)

// MSIBuilder creates MSI packages for the Helix OTA agent.
type MSIBuilder struct {
    outputDir   string
    wixPath     string // Path to WiX Toolset
    version     string
    arch        string // "x64" or "arm64"
    upgradeCode string // GUID that remains constant across
        versions
}

// NewMSIBuilder creates a new MSI builder.
func NewMSIBuilder(outputDir, version, arch string)
        *MSIBuilder {
    return &MSIBuilder{
        outputDir:   outputDir,
        wixPath:     `C:\Program Files (x86)\WiX Toolset
        v4\bin`,
        version:     version,
        arch:        arch,
        upgradeCode: "E9F8A3D2-4B7C-4D5E-8F1A-2C3D4E5F6A7B",
    }
}

// Build generates the MSI package.
func (b *MSIBuilder) Build(ctx context.Context, binDir
        string) (string, error) {
```

```
    // 1. Generate the WiX source file from template
    wxsPath := filepath.Join(b.outputDir, "helix-ota-
        agent.wxs")
    if err := b.generateWXS(wxsPath); err != nil {
        return "", fmt.Errorf("generate WXS: %w", err)
    }

    // 2. Compile the WiX source to an object file
    objPath := filepath.Join(b.outputDir, "helix-ota-
        agent.wixobj")
    candleCmd := exec.CommandContext(ctx,
        filepath.Join(b.wixPath, "candle.exe"),
        wxsPath,
        "-dVersion="+b.version,
        "-dArch="+b.arch,
        "-dBinDir="+binDir,
        "-o", objPath,
    )
    if output, err := candleCmd.CombinedOutput(); err !=
        nil {
        return "", fmt.Errorf("candle failed: %s: %w",
        string(output), err)
    }

    // 3. Link the object file to an MSI package
    msiPath := filepath.Join(b.outputDir,
        fmt.Sprintf("helix-ota-agent-%s-%s.msi",
        b.version, b.arch))
    lightCmd := exec.CommandContext(ctx,
        filepath.Join(b.wixPath, "light.exe"),
        objPath,
        "-o", msiPath,
        "-ext", "WixUtilExtension",
        "-ext", "WixFirewallExtension",
    )
    if output, err := lightCmd.CombinedOutput(); err != nil
        {
        return "", fmt.Errorf("light failed: %s: %w",
        string(output), err)
    }

    return msiPath, nil
}

// generateWXS generates the WiX XML source file.
func (b *MSIBuilder) generateWXS(outputPath string) error {
    tmpl, err := template.New("wix").Parse(wixTemplate)
    if err != nil {
        return fmt.Errorf("parse template: %w", err)
    }

    data := map[string]string{
        "Version":     b.version,
```

```
        "Arch":        b.arch,
        "UpgradeCode": b.upgradeCode,
    }

    f, err := os.Create(outputPath)
    if err != nil {
        return fmt.Errorf("create output file: %w", err)
    }
    defer f.Close()

    return tmpl.Execute(f, data)
}
```

WiX template fragment (helix-ota-agent.wxs):

```
<?xml version="1.0" encoding="UTF-8"?>
<Wix xmlns="http://schemas.microsoft.com/wix/2006/wi"
     xmlns:util="http://schemas.microsoft.com/wix/
        UtilExtension">
  <Product Id="*"
           Name="Helix OTA Agent"
           Language="1033"
           Version="{{.Version}}"
           Manufacturer="Helix OTA Systems"
           UpgradeCode="{{.UpgradeCode}}">

    <Package InstallerVersion="405"
             Compressed="yes"
             InstallScope="perMachine"
             Platform="{{.Arch}}"
             Description="Helix OTA Update Agent
        {{.Version}}" />

    <!-- Major upgrade: uninstall previous version before
        installing new -->
    <MajorUpgrade Schedule="afterInstallInitialize"
                  DowngradeErrorMessage="A newer version of
        Helix OTA Agent is already installed." />

    <MediaTemplate EmbedCab="yes" />

    <Directory Id="TARGETDIR" Name="SourceDir">
      <Directory Id="ProgramFiles64Folder">
        <Directory Id="HELIXFOLDER" Name="Helix OTA">
          <Directory Id="INSTALLDIR" Name="Agent" />
        </Directory>
      </Directory>
    </Directory>

    <DirectoryRef Id="INSTALLDIR">
      <Component Id="ServiceExecutable" Guid="*">
        <File Id="helixOtaAgentExe"
              Source="$(var.BinDir)\helix-ota-agent.exe"
```

```
            KeyPath="yes" />

      <!-- Register as Windows Service -->
      <ServiceInstall Id="HelixOtaService"
                      Name="HelixOtaAgent"
                      DisplayName="Helix OTA Update Agent"
                      Description="Manages over-the-air
      updates"
                      Type="ownProcess"
                      Start="auto"
                      ErrorControl="normal"
                      Account="LocalSystem" />

      <!-- Service failure actions: restart on failure -->
      <ServiceControl Id="HelixOtaServiceStart"
                      Name="HelixOtaAgent"
                      Start="install"
                      Stop="both"
                      Remove="uninstall"
                      Wait="yes" />
    </Component>

    <Component Id="RegistryEntries" Guid="*">
      <RegistryKey Root="HKLM"
                   Key="SOFTWARE\Helix OTA\Agent"
                   ForceCreateOnInstall="yes"
                   ForceDeleteOnUninstall="yes">
        <RegistryValue Type="string" Name="Version"
      Value="{{.Version}}" />
        <RegistryValue Type="string" Name="InstallPath"
      Value="[INSTALLDIR]" />
      </RegistryKey>
    </Component>
  </DirectoryRef>

  <Feature Id="MainFeature" Title="Helix OTA Agent"
      Level="1">
    <ComponentRef Id="ServiceExecutable" />
    <ComponentRef Id="RegistryEntries" />
  </Feature>
 </Product>
</Wix>
```

3.2 MSIX Modern App Packaging

MSIX is Microsoft's modern app packaging format, introduced in Windows 10 1809. It provides containerized installation, automatic updates via the Microsoft Store or private catalogs, and built-in delta updates.

MSIX advantages for OTA:

- **Automatic updates:** MSIX supports app installer files (`.appinstaller`) that define update URLs and frequency, enabling automatic background updates without a custom service.
- **Delta updates:** MSIX supports differential updates, downloading only the changed blocks between versions.
- **Containerization:** MSIX apps run in a lightweight container with predictable file/registry virtualization.
- **Clean uninstall:** MSIX guarantees complete removal with no leftover files or registry entries.

MSIX limitations:

- **Windows version requirement:** MSIX requires Windows 10 1809+ (or Windows 11). Windows 7, 8, and early Windows 10 versions are not supported.
- **Microsoft Store dependency:** Full auto-update requires either the Microsoft Store or a privately hosted app installer feed.
- **Limited system-level access:** MSIX containers restrict access to system resources. A Windows Service cannot run inside an MSIX container.
- **Packaging complexity:** MSIX requires a packaging project in Visual Studio or the MSIX Packaging Tool.

Decision: Because the Helix OTA agent must run as a Windows Service (which is incompatible with MSIX containers), we do not package the OTA agent itself as MSIX. However, we support MSIX as a distribution format for Helix-managed applications. The OTA agent can install MSIX packages via `Add-AppxPackage` as part of an update payload.

```
# Install an MSIX package via the Helix OTA agent
$MsixPath = "C:\HelixOTA\cache\myapp-2.0.0.msix"
$CertPath = "C:\HelixOTA\certs\myapp-signing.cer"

# First, trust the signing certificate (if not already
        trusted)
Import-Certificate -FilePath $CertPath -CertStoreLocation
        Cert:\LocalMachine\TrustedPeople

# Install the MSIX package
Add-AppxPackage -Path $MsixPath -ForceApplicationShutdown

# Verify installation
Get-AppxPackage -Name "Com.Helix.MyApp"
```

3.3 Winget Repository Integration

Windows Package Manager (winget) is Microsoft's official command-line package manager. It is built into Windows 11 and available for Windows 10 via App Installer. Winget is the primary distribution channel for the Helix OTA agent on consumer and small-business devices.

Winget manifest for Helix OTA Agent:

```
# winget-pkgs/manifests/h/HelixOTA/Agent/1.2.0.yaml
PackageIdentifier: HelixOTA.Agent
PackageVersion: 1.2.0
PackageLocale: en-US
Publisher: Helix OTA Systems
PublisherUrl: https://helix.dev
PackageName: Helix OTA Agent
PackageUrl: https://helix.dev/ota-agent
License: Apache-2.0
LicenseUrl: https://opensource.org/licenses/Apache-2.0
Copyright: Copyright (c) 2026 Helix OTA Systems
ShortDescription: Over-the-air update management agent for
      Windows
Description: >
  The Helix OTA Agent manages over-the-air updates for
      applications and
  firmware deployed on Windows devices. It runs as a Windows
      Service and
  provides fleet-level update orchestration, rollback, and
      telemetry.
Tags:
  - ota
  - update
  - fleet-management
  - windows-service
ReleaseNotesUrl: https://helix.dev/releases/ota-agent-1.2.0
InstallerType: msi
Installers:
  - Architecture: x64
    InstallerUrl: https://releases.helix.dev/ota-agent/
      1.2.0/helix-ota-agent-1.2.0-x64.msi
    InstallerSha256:
      4a2f8c1e9d3b7a6f0e5c8d2b1a4f7e3c6d9b2e5a8f1c4d7e0b3a6f9c2e5d8b1
    ProductCode: "{A1B2C3D4-E5F6-7890-ABCD-EF1234567890}"
    Scope: machine
    ElevationRequirement: elevationRequired
    AppsAndFeaturesEntries:
      - DisplayName: Helix OTA Agent
        DisplayVersion: "1.2.0"
        Publisher: Helix OTA Systems
        ProductCode: "{A1B2C3D4-E5F6-7890-ABCD-
        EF1234567890}"
```

```
        UpgradeCode:
        "{E9F8A3D2-4B7C-4D5E-8F1A-2C3D4E5F6A7B}"
  - Architecture: arm64
    InstallerUrl: https://releases.helix.dev/ota-agent/
        1.2.0/helix-ota-agent-1.2.0-arm64.msi
    InstallerSha256:
        b1d8e5c2a9f6d3b7e0c4a1f8d5b2e9c6a3f0d7b4e1c8a5f2d9b6e3c0a7f4d1b
    ProductCode: "{B2C3D4E5-F6A7-8901-BCDE-F12345678901}"
    Scope: machine
    ElevationRequirement: elevationRequired
ManifestType: singleton
ManifestVersion: 1.4.0
```

Winget update check from the Helix OTA server:

```
package winget

import (
    "encoding/json"
    "fmt"
    "net/http"
    "strings"
)

// WingetManifestClient queries the winget REST API for
        package information.
type WingetManifestClient struct {
    baseURL    string
    httpClient *http.Client
}

// NewWingetManifestClient creates a client for the winget
        package repository.
func NewWingetManifestClient() *WingetManifestClient {
    return &WingetManifestClient{
        baseURL:    "https://api.winget.microsoft.com/v1",
        httpClient: &http.Client{},
    }
}

// GetLatestVersion queries the latest available version of
        a package.
func (c *WingetManifestClient) GetLatestVersion(packageID
        string) (string, error) {
    url := fmt.Sprintf("%s/packageManifests/%s", c.baseURL,
        packageID)
    resp, err := c.httpClient.Get(url)
    if err != nil {
        return "", fmt.Errorf("winget API request failed:
        %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
```

```
        return "", fmt.Errorf("winget API returned status
        %d", resp.StatusCode)
    }

    var manifest struct {
        Data struct {
            Versions []struct {
                PackageVersion string
        `json:"packageVersion"`
            } `json:"versions"`
        } `json:"Data"`
    }

    if err := json.NewDecoder(resp.Body).Decode(&manifest);
        err != nil {
        return "", fmt.Errorf("failed to decode manifest:
        %w", err)
    }

    if len(manifest.Data.Versions) == 0 {
        return "",
        fmt.Errorf("no versions found for package %s",
        packageID)
    }

    return manifest.Data.Versions[0].PackageVersion, nil
}
```

3.4 Custom Update Catalog Format

For deployments that do not use Winget, WSUS, or Intune, Helix OTA uses a custom update catalog format. This catalog is a JSON document hosted on the Helix OTA server that describes available updates, their artifacts, and installation instructions.

```
package catalog

import (
    "encoding/json"
    "fmt"
    "time"
)

// UpdateCatalog represents the Helix OTA custom update
        catalog.
type UpdateCatalog struct {
    SchemaVersion string         `json:"schema_version"`
    GeneratedAt   string         `json:"generated_at"`
    Channel       string         `json:"channel"` //
        "stable", "beta", "canary"
    Updates       []UpdateEntry  `json:"updates"`
}
```

```
// UpdateEntry describes a single available update.
type UpdateEntry struct {
    ID              string            `json:"id"`
    Version         string            `json:"version"`
    DisplayName     string            `json:"display_name"`
    Description     string            `json:"description"`
    ReleaseNotesURL string
        `json:"release_notes_url"`
    PublishedAt     string            `json:"published_at"`
    ExpiresAt       string
        `json:"expires_at,omitempty"`
    MinimumOSVersion string
        `json:"minimum_os_version"` // e.g., "10.0.19041"
    MaximumOSVersion string
        `json:"maximum_os_version,omitempty"`
    Architecture    []string
        `json:"architecture"` // ["x64", "arm64"]
    Artifacts       []ArtifactEntry   `json:"artifacts"`
    Requirements    []Requirement
        `json:"requirements,omitempty"`
    PostInstall     []PostInstallStep
        `json:"post_install,omitempty"`
}

// ArtifactEntry describes a downloadable artifact.
type ArtifactEntry struct {
    Filename    string `json:"filename"`
    URL         string `json:"url"`
    SizeBytes   int64  `json:"size_bytes"`
    HashSHA256  string `json:"hash_sha256"`
    HashSHA1    string `json:"hash_sha1,omitempty"`
    Format      string `json:"format"` // "msi", "msix",
        "exe", "zip"
    DeltaFrom   string `json:"delta_from,omitempty"`  //
        Version this is a delta from
    DeltaFormat string `json:"delta_format,omitempty"` //
        "bsdiff", "zstd-patch"
}

// Requirement describes a prerequisite for installation.
type Requirement struct {
    Type  string `json:"type"`  // "dotnet", "vcrt",
        "disk_space", "memory"
    Value string `json:"value"` // e.g., "8.0", "14.38",
        "500MB", "2GB"
}

// PostInstallStep describes a post-installation action.
type PostInstallStep struct {
    Type     string `json:"type"`     // "reboot",
        "service_start", "registry", "script"
    Command  string `json:"command,omitempty"`
    DelaySec int    `json:"delay_sec,omitempty"`
```

```
}

// GenerateCatalog creates an update catalog for a given
        channel.
func GenerateCatalog(channel string, updates []UpdateEntry)
        ([]byte, error) {
    catalog := UpdateCatalog{
        SchemaVersion: "1.0",
        GeneratedAt:
        time.Now().UTC().Format(time.RFC3339),
        Channel:       channel,
        Updates:       updates,
    }

    data, err := json.MarshalIndent(catalog, "", "  ")
    if err != nil {
        return nil, fmt.Errorf("failed to marshal catalog:
        %w", err)
    }

    return data, nil
}
```

Example catalog output:

```
{
  "schema_version": "1.0",
  "generated_at": "2026-03-05T14:30:00Z",
  "channel": "stable",
  "updates": [
    {
      "id": "helix-ota-agent-1.2.0",
      "version": "1.2.0",
      "display_name": "Helix OTA Agent 1.2.0",
      "description": "Adds Windows OTA support, BITS
        download integration, and MSI packaging",
      "release_notes_url": "https://helix.dev/releases/ota-
        agent-1.2.0",
      "published_at": "2026-03-05T00:00:00Z",
      "minimum_os_version": "10.0.19041",
      "architecture": ["x64", "arm64"],
      "artifacts": [
        {
          "filename": "helix-ota-agent-1.2.0-x64.msi",
          "url": "https://ota.helix.dev/artifacts/helix-ota-
        agent-1.2.0-x64.msi",
          "size_bytes": 12582912,
          "hash_sha256":
        "4a2f8c1e9d3b7a6f0e5c8d2b1a4f7e3c6d9b2e5a8f1c4d7e0b3a6f9c2e5d8b1",
          "format": "msi"
        },
        {
          "filename": "helix-ota-agent-1.2.0-x64-delta-
        from-1.1.0.patch",
```

```
          "url": "https://ota.helix.dev/artifacts/helix-ota-
        agent-1.2.0-x64-delta-from-1.1.0.patch",
          "size_bytes": 3145728,
          "hash_sha256":
        "c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d",
          "format": "msi",
          "delta_from": "1.1.0",
          "delta_format": "bsdiff"
        }
      ],
      "requirements": [
        {"type": "vcrt", "value": "14.38"},
        {"type": "disk_space", "value": "200MB"}
      ],
      "post_install": [
        {"type": "service_start", "command":
        "HelixOtaAgent"},
        {"type": "reboot", "delay_sec": 300}
      ]
    }
  ]
}
```

3.5 Digital Signing with Authenticode

All Helix OTA installers must be signed with an Authenticode certificate. Windows uses Authenticode signatures to verify the publisher identity and integrity of executable files. Without a valid signature, Windows Defender SmartScreen will display a warning to users, and enterprise deployment tools may refuse to install the package.

Signing workflow:

```
package signing

import (
    "fmt"
    "os/exec"
    "path/filepath"
)

// AuthenticodeSigner signs Windows executables and MSI
        packages.
type AuthenticodeSigner struct {
    signToolPath string
    certPath     string
    certPassword string
    timestampURL string
}
```

```
// NewAuthenticodeSigner creates a new code signer.
func NewAuthenticodeSigner(certPath, certPassword string)
        *AuthenticodeSigner {
    return &AuthenticodeSigner{
        signToolPath: `C:\Program Files (x86)\Windows
        Kits\10\bin\10.0.26100.0\x64\signtool.exe`,
        certPath:     certPath,
        certPassword: certPassword,
        timestampURL: "http://timestamp.digicert.com",
    }
}

// SignFile signs a single file with Authenticode.
func (s *AuthenticodeSigner) SignFile(filePath string)
        error {
    args := []string{
        "sign",
        "/fd", "SHA256",       // File digest algorithm
        "/a",                   // Use best available
        certificate
        "/f", s.certPath,       // Certificate file
        "/p", s.certPassword,   // Certificate password
        "/tr", s.timestampURL,  // RFC 3161 timestamp server
        "/td", "SHA256",        // Timestamp digest
        algorithm
        filePath,
    }

    cmd := exec.Command(s.signToolPath, args...)
    output, err := cmd.CombinedOutput()
    if err != nil {
        return fmt.Errorf("signtool failed: %s: %w",
        string(output), err)
    }

    return nil
}

// SignMSI signs an MSI package (signs all embedded CAB
        files and the MSI itself).
func (s *AuthenticodeSigner) SignMSI(msiPath string) error {
    // Sign the MSI file — signtool signs the entire package
    return s.SignFile(msiPath)
}

// VerifySignature verifies the Authenticode signature on a
        file.
func (s *AuthenticodeSigner) VerifySignature(filePath
        string) error {
    args := []string{
        "verify",
        "/pa",  // Use the Default Authentication
        Verification Policy
```

```
        "/all", // Verify all signatures (for multi-signed
        files)
        filePath,
    }

    cmd := exec.Command(s.signToolPath, args...)
    output, err := cmd.CombinedOutput()
    if err != nil {
        return fmt.Errorf("signature verification failed:
        %s: %w", string(output), err)
    }

    return nil
}
```

---

4. Windows-Specific Security

4.1 Code Signing Requirements

Windows has stringent code signing requirements that go beyond what is typical on Linux or Android. The Helix OTA agent and all update artifacts must be properly signed to ensure:

1. **Publisher verification:** Users and administrators can verify that the update comes from Helix OTA Systems.
2. **Integrity verification:** Any tampering with the update artifact after signing is detected.
3. **SmartScreen reputation:** Properly signed applications build reputation over time, reducing SmartScreen warnings.
4. **Enterprise deployment:** Group Policy can be configured to only install software signed by specific publishers.

Certificate requirements:

Property	Requirement
Certificate type	Extended Validation (EV) Code Signing Certificate
Key length	RSA 2048-bit minimum; RSA 4096-bit recommended
Key storage	Hardware security module (HSM) or USB token (required for EV)
Timestamp	RFC 3161 timestamp from a trusted timestamp authority

Property	Requirement
Digest algorithm	SHA-256 minimum; SHA-384 recommended
Chain	Must chain to a trusted root CA in the Windows trust store

PowerShell verification script:

```
# Verify the Authenticode signature of a file
function Test-HelixOtaSignature {
    param(
        [Parameter(Mandatory)]
        [string]$FilePath,

        [Parameter(Mandatory)]
        [string]$ExpectedPublisher
    )

    $Signature = Get-AuthenticodeSignature -FilePath
        $FilePath

    if ($Signature.Status -ne 'Valid') {
        Write-Error "Signature status: $($Signature.Status)
        - $($Signature.StatusMessage)"
        return $false
    }

    if ($Signature.SignerCertificate.Subject -notmatch
        "CN=$ExpectedPublisher") {
        Write-Error "Unexpected publisher: $
        ($Signature.SignerCertificate.Subject)"
        return $false
    }

    # Verify the certificate chain
    $Chain = New-Object
        System.Security.Cryptography.X509Certificates.X509Chain
    $Chain.ChainPolicy.RevocationMode =
        [System.Security.Cryptography.X509Certificates.X509RevocationMode]::On
    $Chain.ChainPolicy.RevocationFlag =
        [System.Security.Cryptography.X509Certificates.X509RevocationFlag]::En

    if (-not $Chain.Build($Signature.SignerCertificate)) {
        Write-Error "Certificate chain validation failed:"
        foreach ($Status in $Chain.ChainStatus) {
            Write-Error "  $($Status.StatusInformation)"
        }
        return $false
    }
```

```
        # Verify timestamp (signature should be valid even
        after cert expiry)
    if ($Signature.TimeStamperCertificate) {
        Write-Host "Timestamp: $
        ($Signature.TimeStamperCertificate.NotBefore) -
        Valid"
    }

    Write-Host "Signature verification passed"
    Write-Host "  Publisher: $
        ($Signature.SignerCertificate.Subject)"
    Write-Host "  Valid from: $
        ($Signature.SignerCertificate.NotBefore)"
    Write-Host "  Valid until: $
        ($Signature.SignerCertificate.NotAfter)"
    Write-Host "  Algorithm: $
        ($Signature.HashAlgorithm.FriendlyName)"

    return $true
}
```

4.2 Windows Defender SmartScreen

SmartScreen is a cloud-based reputation service that warns users about potentially dangerous downloads. New applications without established reputation will trigger a SmartScreen warning.

SmartScreen reputation building strategy:

1. **EV Code Signing Certificate:** Applications signed with an EV certificate immediately establish reputation, bypassing the SmartScreen warning period. This is the primary reason we require an EV certificate.
2. **Gradual rollout:** Initially deploy the Helix OTA agent only to managed enterprise devices (where SmartScreen can be disabled via Group Policy). As the application builds organic download reputation, expand to consumer devices.
3. **Microsoft Partner Center submission:** Submit the Helix OTA agent to Microsoft for analysis. Once approved, the application is added to the SmartScreen allow list.

SmartScreen Group Policy bypass (enterprise deployment):

```
# Disable SmartScreen for files from trusted publishers
        (enterprise GPO)
# Path: Computer Configuration > Administrative Templates >
        Windows Components >
```

```
# Windows Defender SmartScreen > Explorer > Configure
    Windows Defender SmartScreen

$RegPath = "HKLM:
    \SOFTWARE\Policies\Microsoft\Windows\System"
Set-ItemProperty -Path $RegPath -Name "EnableSmartScreen"
    -Value 1 -Type DWord
Set-ItemProperty -Path $RegPath -Name
    "ShellSmartScreenLevel" -Value "Warn" -Type String

# Allow apps from trusted publishers only
$SmartScreenPath = "HKLM:
    \SOFTWARE\Policies\Microsoft\Windows\SmartScreen"
New-Item -Path $SmartScreenPath -Force | Out-Null
Set-ItemProperty -Path $SmartScreenPath -Name
    "TrustedPublisherList" -Value "Helix OTA Systems"
    -Type String
```

4.3 UAC Handling During Installation

User Account Control (UAC) is a Windows security feature that prevents unauthorized changes to the system by requiring explicit consent for actions that require administrator privileges.

UAC considerations for Helix OTA:

1. **Silent installation:** The MSI installer must support silent installation (`msiexec /i helix-ota-agent.msi /qn`) for enterprise deployment. UAC elevation occurs automatically when the installer is launched by a system management tool (Intune, MECM, Group Policy) running as SYSTEM.
2. **Self-updating:** When the OTA agent updates itself, it must request elevation. Since the service already runs as LocalSystem, self-updates do not trigger a UAC prompt for the user. However, the update must be performed carefully to avoid leaving the system in an inconsistent state.
3. **Application manifest:** The tray application and CLI must embed a UAC manifest requesting `asInvoker` (not `requireAdministrator`) to avoid unnecessary UAC prompts when the user launches them.

Application manifest embedding:

```
//go:embed app.manifest
var appManifest []byte

// The manifest is embedded via a .syso file compiled with
    rsrc:
// rsrc -manifest app.manifest -o rsrc.syso
```

app.manifest:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<assembly xmlns="urn:schemas-microsoft-com:asm.v1"
        manifestVersion="1.0">
  <trustInfo xmlns="urn:schemas-microsoft-com:asm.v3">
    <security>
      <requestedPrivileges>
        <!-- asInvoker: run with the same privileges as the
        parent process -->
        <requestedExecutionLevel level="asInvoker"
        uiAccess="false" />
      </requestedPrivileges>
    </security>
  </trustInfo>
  <!-- Declare DPI awareness for correct display scaling -->
  <application xmlns="urn:schemas-microsoft-com:asm.v3">
    <windowsSettings>
      <dpiAware xmlns="http://schemas.microsoft.com/SMI/
        2005/WindowsSettings">true</dpiAware>
      <dpiAwareness xmlns="http://schemas.microsoft.com/SMI/
        2016/WindowsSettings">PerMonitorV2</dpiAwareness>
    </windowsSettings>
  </application>
  <!-- Declare Windows 10/11 compatibility -->
  <compatibility xmlns="urn:schemas-microsoft-
        com:compatibility.v1">
    <application>
      <supportedOS Id="{8e0f7a12-bfb3-4fe8-
        b9a5-48fd50a15a9a}" /> <!-- Windows 10 -->
    </application>
  </compatibility>
</assembly>
```

4.4 Secure Boot and TPM Integration

Modern Windows devices ship with Secure Boot and TPM (Trusted Platform Module) enabled. These features provide hardware-rooted security that the Helix OTA agent must respect and leverage.

Secure Boot considerations:

- Secure Boot ensures that only signed boot loaders and kernels can execute. Our OTA agent does not modify the boot chain (we update applications, not the OS), so Secure Boot does not directly affect our operation.
- However, if a Helix-managed update includes a kernel-mode driver, the driver must be WHQL-signed and submitted to Microsoft for attestation signing. Self-signed kernel drivers are blocked by Secure Boot.

TPM integration:

The TPM provides hardware-backed key storage and measured boot. The Helix OTA agent can leverage the TPM for:

1. **Device identity:** Each device has a unique TPM-bound attestation key that can be used for device authentication to the OTA server, replacing software-generated device certificates.
2. **Secure credential storage:** The TPM can seal the OTA server's client certificate, ensuring it cannot be extracted even by an attacker with physical access.
3. **Measured boot attestation:** The agent can report the device's measured boot log to the OTA server, allowing the server to verify the device's boot integrity before delivering sensitive updates.

TPM device attestation in Go:

```
package tpm

import (
    "fmt"

    "github.com/google/go-tpm/tpm2"
    "github.com/google/go-tpm/tpmutil"
)

const tpmDevicePath = `\\.\TPM0`

// TPMAttestor provides TPM-based device attestation.
type TPMAttestor struct {
    tpm *tpm2.TPM
}

// NewTPMAttestor opens a connection to the TPM.
func NewTPMAttestor() (*TPMAttestor, error) {
    tpm, err := tpm2.OpenTPM(tpmDevicePath)
    if err != nil {
        return nil, fmt.Errorf("failed to open TPM: %w",
        err)
    }
    return &TPMAttestor{tpm: tpm}, nil
}

// GetDeviceID returns a TPM-bound device identifier.
func (t *TPMAttestor) GetDeviceID() (string, error) {
    // Read the Endorsement Key (EK) certificate from the
        TPM
    ekCert, err := t.readEKCertificate()
    if err != nil {
```

```
        return "", fmt.Errorf("failed to read EK
        certificate: %w", err)
    }

    // Hash the EK public key to create a stable device
        identifier
    fingerprint := sha256Fingerprint(ekCert.PublicKey)

    return fingerprint, nil
}

// AttestDevice performs remote attestation of the device's
        boot state.
func (t *TPMAttestor) AttestDevice(nonce []byte)
        (*AttestationReport, error) {
    // 1. Get the Attestation Identity Key (AIK)
    aikHandle, aikPub, err := t.getAIK()
    if err != nil {
        return nil, fmt.Errorf("failed to get AIK: %w", err)
    }

    // 2. Quote the PCRs (Platform Configuration Registers)
        using the AIK
    // This produces a signed statement of the current boot
        state
    pcrSelection := tpm2.PCRSelection{
        Hash: tpm2.AlgSHA256,
        PCRs: []int{0, 1, 2, 3, 4, 5, 6,
        7}, // Boot-related PCRs
    }

    quote, signature, err := tpm2.Quote(
        t.tpm,
        aikHandle,
        "",        // Password (empty if no password set)
        nonce,     // Qualifying data (nonce from server)
        pcrSelection,
        tpm2.AlgSHA256,
    )
    if err != nil {
        return nil, fmt.Errorf("TPM quote failed: %w", err)
    }

    // 3. Read the PCR values for verification
    pcrValues, err := tpm2.ReadPCRs(t.tpm, pcrSelection)
    if err != nil {
        return nil, fmt.Errorf("failed to read PCRs: %w",
        err)
    }

    return &AttestationReport{
        Quote:     quote,
        Signature: signature,
```

```
        AIKPublic: aikPub,
        PCRValues: pcrValues,
        Nonce:     nonce,
    }, nil
}

// AttestationReport contains the TPM attestation data.
type AttestationReport struct {
    Quote     []byte
    Signature []byte
    AIKPublic []byte
    PCRValues map[uint][]byte
    Nonce     []byte
}

func (t *TPMAttestor) readEKCertificate()
        (*x509.Certificate, error) {
    // Implementation reads the EK certificate from NV index
        0x01C00002
    return nil, fmt.Errorf("not implemented")
}

func (t *TPMAttestor) getAIK() (tpmutil.Handle, []byte,
        error) {
    // Implementation creates or loads a persistent AIK
    return 0, nil, fmt.Errorf("not implemented")
}

// Close releases the TPM connection.
func (t *TPMAttestor) Close() error {
    return t.tpm.Close()
}
```

4.5 Windows Credential Store for Certificates

The Helix OTA agent stores its client certificate (used for mTLS authentication to the OTA server) in the Windows Certificate Store rather than as a flat file on disk. This provides:

1. **Access control:** The certificate's private key is protected by the Windows DPAPI (Data Protection API), which encrypts it with the user's or computer's credentials.
2. **Audit logging:** Access to the certificate can be audited via Windows Event Log.
3. **Central management:** Enterprise administrators can deploy and rotate certificates via Group Policy or Intune.

Certificate store operations in Go:

```
package certstore

import (
    "crypto/tls"
    "crypto/x509"
    "fmt"
    "unsafe"

    "golang.org/x/sys/windows"
)

const (
    storeName    =
        "MY"                            // Personal
        certificate store
    storeLocation =
        windows.CERT_SYSTEM_STORE_LOCAL_MACHINE // Machine-
        level store
)

// CertificateManager manages certificates in the Windows
        Certificate Store.
type CertificateManager struct{}

// LoadClientCertificate loads the Helix OTA client
        certificate from the
// Windows Certificate Store for use in mTLS connections.
func (cm *CertificateManager) LoadClientCertificate()
        (tls.Certificate, error) {
    // Open the Local Machine "My" (Personal) certificate
        store
    store, err := windows.CertOpenStore(
        windows.CERT_STORE_PROV_SYSTEM,
        0,
        0,
        windows.CERT_SYSTEM_STORE_LOCAL_MACHINE,
        windows.StringToUTF16Ptr(storeName),
    )
    if err != nil {
        return tls.Certificate{},
        fmt.Errorf("failed to open certificate store: %w",
        err)
    }
    defer windows.CertCloseStore(store, 0)

    // Enumerate certificates to find the Helix OTA client
        certificate
    var certContext *windows.CertContext
    for {
        certContext, err =
        windows.CertEnumCertificatesInStore(store,
        certContext)
        if err != nil {
            break
```

```
        }

        // Check if this is our client certificate by
        looking at the Subject
        cert, err :=
        x509.ParseCertificate(certContext.EncodedCert)
        if err != nil {
            continue
        }

        if cert.Subject.CommonName == "helix-ota-client" {
            // Found our certificate - load it as a
        tls.Certificate
            // This requires access to the private key,
        which is
            // associated with the certificate in the
        Windows store
            return cm.loadTLSCertificate(certContext)
        }
    }

    return tls.Certificate{}, fmt.Errorf("helix-ota-client
        certificate not found in Windows Certificate
        Store")
}

// ImportCertificate imports a PFX file into the Windows
        Certificate Store.
// This is called during initial device provisioning.
func (cm *CertificateManager) ImportCertificate(pfxPath,
        password string) error {
    // Read the PFX file
    pfxData, err := os.ReadFile(pfxPath)
    if err != nil {
        return fmt.Errorf("failed to read PFX file: %w",
        err)
    }

    // Import the PFX into the Local Machine Personal store
    var pfxPara windows.CryptDataBlob
    pfxPara.Size = uint32(len(pfxData))
    pfxPara.Data = &pfxData[0]

    store, err := windows.CertOpenStore(
        windows.CERT_STORE_PROV_SYSTEM,
        0,
        0,
        windows.CERT_SYSTEM_STORE_LOCAL_MACHINE,
        windows.StringToUTF16Ptr(storeName),
    )
    if err != nil {
        return
        fmt.Errorf("failed to open certificate store: %w",
        err)
```

```
    }
    defer windows.CertCloseStore(store, 0)

    // Use PFXImportCertStore to import the PFX
    // Then add the certificate to our store
    // In practice, use cryptoapi.PFXImportCertStore for
        full implementation
    return fmt.Errorf("PFX import requires full cryptoapi
        implementation")
}
```

PowerShell certificate management:

```
# Import the Helix OTA client certificate into the Local
        Machine store
$PfxPath = "C:\HelixOTA\provisioning\client-cert.pfx"
$Password = Read-Host -AsSecureString -Prompt "Enter PFX
        password"

Import-PfxCertificate -FilePath $PfxPath `
    -CertStoreLocation Cert:\LocalMachine\My `
    -Password $Password

# Verify the certificate was imported
$Cert = Get-ChildItem Cert:\LocalMachine\My | Where-Object {
    $_.Subject -match "helix-ota-client"
}

if ($Cert) {
    Write-Host "Certificate imported successfully"
    Write-Host "  Thumbprint: $($Cert.Thumbprint)"
    Write-Host "  Subject: $($Cert.Subject)"
    Write-Host "  NotBefore: $($Cert.NotBefore)"
    Write-Host "  NotAfter: $($Cert.NotAfter)"
} else {
    Write-Error "Certificate not found after import"
}

# Grant the Helix OTA service account read access to the
        private key
$Rule = New-Object
        System.Security.AccessControl.FileSystemAccessRule(
    "NT AUTHORITY\SYSTEM", "Read", "Allow"
)
$CertPrivKeyPath = "$
        ($Cert.PrivateKey.CspKeyContainerInfo.UniqueKeyContainerName)"
$KeyPath = "C:\ProgramData\Microsoft\Crypto\RSA\MachineKeys\
        $CertPrivKeyPath"
$Acl = Get-Acl $KeyPath
$Acl.AddAccessRule($Rule)
Set-Acl $KeyPath $Acl
```

---

5. Windows Adapter Implementation

5.1 OSAdapter Interface for Windows

The Windows adapter implements the same `OSAdapter` interface defined in the Helix OTA core module, providing Windows-specific implementations for update checking, downloading, installation, verification, and rollback.

```
package windowsadapter

import (
    "context"
    "fmt"
    "os"
    "os/exec"
    "path/filepath"
    "time"

    "github.com/helix-ota/core/adapter"
)

// WindowsAdapter implements adapter.OSAdapter for Microsoft
        Windows.
type WindowsAdapter struct {
    config     WindowsAdapterConfig
    bitsMgr    *bits.BITSManager
    eventLog   *eventlog.Logger
    certMgr    *certstore.CertificateManager
    tpm        *tpm.TPMAttestor
    ipcServer  *ipc.PipeServer
}

// WindowsAdapterConfig holds Windows-specific adapter
        configuration.
type WindowsAdapterConfig struct {
    // Installation paths
    InstallDir    string `json:"install_dir"
        yaml:"install_dir"`       // e.g., "C:\Program
        Files\Helix OTA\Agent"
    CacheDir      string `json:"cache_dir"
        yaml:"cache_dir"`           // e.g., "C:
        \ProgramData\Helix OTA\Cache"
    ConfigDir     string `json:"config_dir"
        yaml:"config_dir"`         // e.g., "C:
        \ProgramData\Helix OTA\Config"
    LogDir        string `json:"log_dir"
        yaml:"log_dir"`               // e.g., "C:
        \ProgramData\Helix OTA\Logs"

    // Server connection
```

```
    ServerURL     string `json:"server_url"
        yaml:"server_url"`         // e.g., "https://
        ota.helix.dev"
    CertThumbprint string `json:"cert_thumbprint"
        yaml:"cert_thumbprint"` // Client cert thumbprint

    // Update behavior
    CheckInterval string `json:"check_interval"
        yaml:"check_interval"` // e.g., "1h"
    DownloadPriority int `json:"download_priority"
        yaml:"download_priority"` // BITS priority: 0-3
    AutoReboot    bool   `json:"auto_reboot"
        yaml:"auto_reboot"`       // Auto-reboot after
        update if required
    RebootDeadline string `json:"reboot_deadline"
        yaml:"reboot_deadline"` // e.g., "4h"

    // Rollback
    MaxRollbackVersions int `json:"max_rollback_versions"
        yaml:"max_rollback_versions"`
    BackupDir      string `json:"backup_dir"
        yaml:"backup_dir"`       // e.g., "C:
        \ProgramData\Helix OTA\Backups"
}

// CheckForUpdate queries the Helix OTA server for available
        updates.
func (w *WindowsAdapter) CheckForUpdate(
    ctx context.Context,
    device adapter.Device,
) (*adapter.UpdateInfo, error) {
    w.eventLog.Info(eventlog.EventIDUpdateCheckStart,
        "Checking for updates from %s", w.config.ServerURL)

    // 1. Build the device report with current version info
    report := w.buildDeviceReport(device)

    // 2. Query the OTA server for available updates
    updateInfo, err := w.queryServer(ctx, report)
    if err != nil {
        w.eventLog.Warning(eventlog.EventIDUpdateNotFound,
            "Update check failed: %v", err)
        return nil, fmt.Errorf("update check failed: %w",
        err)
    }

    if updateInfo == nil {
        w.eventLog.Info(eventlog.EventIDUpdateNotFound,
            "No updates available")
        return nil, nil
    }

    w.eventLog.Info(eventlog.EventIDUpdateAvailable,
```

```
        "Update available: %s (%s)", updateInfo.Version,
        updateInfo.Description)

    return updateInfo, nil
}

// ApplyUpdate downloads and installs the update.
func (w *WindowsAdapter) ApplyUpdate(
    ctx context.Context,
    device adapter.Device,
    artifact adapter.Artifact,
) error {
    // 1. Create a backup of the current version for
        rollback
    if err := w.createBackup(ctx); err != nil {
        return fmt.Errorf("backup failed: %w", err)
    }

    // 2. Download the update artifact using BITS
    localPath, err := w.downloadArtifact(ctx, artifact)
    if err != nil {
        return fmt.Errorf("download failed: %w", err)
    }

    // 3. Verify the artifact signature and hash
    if err := w.verifyArtifact(ctx, localPath, artifact);
        err != nil {
        return fmt.Errorf("verification failed: %w", err)
    }

    // 4. Install the update
    if err := w.installArtifact(ctx, localPath, artifact);
        err != nil {
        return fmt.Errorf("installation failed: %w", err)
    }

    // 5. Notify the tray application
    w.ipcServer.Broadcast(ipc.IPCMessage{
        Type:      ipc.MsgUpdateApplied,
        Timestamp: time.Now().UTC().Format(time.RFC3339),
        Payload:   map[string]interface{}{"version":
        artifact.Version},
    })

    return nil
}

// VerifyUpdate confirms that the update was applied
        correctly.
func (w *WindowsAdapter) VerifyUpdate(ctx context.Context,
        device adapter.Device) error {
    // 1. Check the installed version in the registry
    installedVersion, err := w.getInstalledVersion()
```

```
    if err != nil {
        return
        fmt.Errorf("failed to read installed version: %w",
        err)
    }

    // 2. Verify the service is running
    svcStatus, err := w.queryServiceStatus(ctx)
    if err != nil || svcStatus != "Running" {
        return
        fmt.Errorf("service not running after update:
        status=%s err=%v", svcStatus, err)
    }

    // 3. Run post-update health checks
    if err := w.runHealthChecks(ctx); err != nil {
        return fmt.Errorf("post-update health check failed:
        %w", err)
    }

    return nil
}

// Rollback reverts to the previous version.
func (w *WindowsAdapter) Rollback(ctx context.Context,
        device adapter.Device) error {
    w.eventLog.Info(eventlog.EventIDRollbackStart,
        "Initiating rollback to previous version")

    // 1. Stop the current service
    if err := w.stopService(ctx); err != nil {
        w.eventLog.Warning(eventlog.EventIDRollbackFailed,
            "Failed to stop service during rollback: %v",
        err)
    }

    // 2. Restore the backup
    if err := w.restoreBackup(ctx); err != nil {
        w.eventLog.Error(eventlog.EventIDRollbackFailed,
            "Failed to restore backup: %v", err)
        return fmt.Errorf("rollback failed: %w", err)
    }

    // 3. Start the service with the previous version
    if err := w.startService(ctx); err != nil {
        w.eventLog.Error(eventlog.EventIDRollbackFailed,
            "Failed to start service after rollback: %v",
        err)
        return fmt.Errorf("failed to start service after
        rollback: %w", err)
    }

    w.eventLog.Info(eventlog.EventIDRollbackComplete,
```

```
        "Rollback completed successfully")

    return nil
}

// ReportStatus returns the current update status.
func (w *WindowsAdapter) ReportStatus(ctx context.Context,
        device adapter.Device) (*adapter.UpdateStatus,
        error) {
    version, _ := w.getInstalledVersion()
    svcStatus, _ := w.queryServiceStatus(ctx)
    pendingReboot := w.isRebootPending()

    return &adapter.UpdateStatus{
        State:         w.mapServiceStatusToState(svcStatus),
        CurrentVersion: version,
        Metadata: map[string]interface{}{
            "service_status": svcStatus,
            "pending_reboot": pendingReboot,
            "os_version":     w.getOSVersion(),
            "install_dir":    w.config.InstallDir,
        },
    }, nil
}
```

5.2 Registry-Based Configuration

The Windows Registry is the canonical configuration store for the Helix OTA agent on Windows. Configuration is stored under `HKLM\SOFTWARE\Helix OTA\Agent` and managed through the adapter's registry client.

```
package registry

import (
    "fmt"
    "golang.org/x/sys/windows/registry"
)

const basePath = `SOFTWARE\Helix OTA\Agent`

// RegistryConfig reads and writes configuration from the
        Windows Registry.
type RegistryConfig struct{}

// Load reads the adapter configuration from the registry.
func (r *RegistryConfig) Load() (*WindowsAdapterConfig,
        error) {
    k, err := registry.OpenKey(registry.LOCAL_MACHINE,
        basePath, registry.READ)
    if err != nil {
```

```
            return nil,
                fmt.Errorf("failed to open registry key: %w", err)
        }
        defer k.Close()

        config := &WindowsAdapterConfig{}

        config.InstallDir, _, err =
            k.GetStringValue("InstallPath")
        if err != nil {
            config.InstallDir = `C:\Program Files\Helix
            OTA\Agent`
        }

        config.CacheDir, _, err = k.GetStringValue("CacheDir")
        if err != nil {
            config.CacheDir = `C:\ProgramData\Helix OTA\Cache`
        }

        config.ServerURL, _, err = k.GetStringValue("ServerURL")
        if err != nil {
            return nil, fmt.Errorf("ServerURL not configured in
            registry")
        }

        config.CheckInterval, _, err =
            k.GetStringValue("CheckInterval")
        if err != nil {
            config.CheckInterval = "1h"
        }

        config.AutoReboot, _, err =
            k.GetBooleanValue("AutoReboot")
        if err != nil {
            config.AutoReboot = false
        }

        config.DownloadPriority, _, err =
            k.GetIntegerValue("DownloadPriority")
        if err != nil {
            config.DownloadPriority = 2 // BITS normal priority
        }

        return config, nil
    }

    // Save writes the adapter configuration to the registry.
    func (r *RegistryConfig) Save(config *WindowsAdapterConfig)
            error {
        k, _, err := registry.CreateKey(registry.LOCAL_MACHINE,
            basePath, registry.WRITE)
        if err != nil {
```

```
            return fmt.Errorf("failed to create registry key:
            %w", err)
        }
        defer k.Close()

        if err := k.SetStringValue("InstallPath",
            config.InstallDir); err != nil {
            return err
        }
        if err := k.SetStringValue("CacheDir",
            config.CacheDir); err != nil {
            return err
        }
        if err := k.SetStringValue("ServerURL",
            config.ServerURL); err != nil {
            return err
        }
        if err := k.SetStringValue("CheckInterval",
            config.CheckInterval); err != nil {
            return err
        }
        if err := k.SetBooleanValue("AutoReboot",
            config.AutoReboot); err != nil {
            return err
        }
        if err := k.SetDWordValue("DownloadPriority",
            uint32(config.DownloadPriority)); err != nil {
            return err
        }

        return nil
    }
```

Registry configuration via PowerShell:

```
    # Configure the Helix OTA agent via the Windows Registry
    $RegPath = "HKLM:\SOFTWARE\Helix OTA\Agent"

    # Set the OTA server URL
    Set-ItemProperty -Path $RegPath -Name "ServerURL" -Value
            "https://ota.helix.dev" -Type String

    # Set the update check interval (Go duration format)
    Set-ItemProperty -Path $RegPath -Name "CheckInterval"
            -Value "30m" -Type String

    # Enable automatic reboot after updates
    Set-ItemProperty -Path $RegPath -Name "AutoReboot" -Value 1
            -Type DWord

    # Set BITS download priority (0=Foreground, 1=High,
            2=Normal, 3=Low)
    Set-ItemProperty -Path $RegPath -Name "DownloadPriority"
            -Value 1 -Type DWord
```

```
# Configure the reboot deadline
Set-ItemProperty -Path $RegPath -Name "RebootDeadline"
        -Value "4h" -Type String

# Read current configuration
Get-ItemProperty -Path $RegPath | Format-List
```

5.3 PowerShell Integration for Update Operations

The Helix OTA agent uses PowerShell for various Windows-specific operations that are more easily expressed as scripts than as native Go code. These include MSI installation, service management, and system queries.

```
package powershell

import (
    "bytes"
    "context"
    "fmt"
    "os/exec"
)

// Runner executes PowerShell scripts and commands.
type Runner struct{}

// Run executes a PowerShell script and returns its output.
func (r *Runner) Run(ctx context.Context, script string)
        (string, error) {
    cmd := exec.CommandContext(ctx,
        "powershell.exe",
        "-NoProfile",
        "-NonInteractive",
        "-ExecutionPolicy", "Bypass",
        "-Command", script,
    )

    var stdout, stderr bytes.Buffer
    cmd.Stdout = &stdout
    cmd.Stderr = &stderr

    if err := cmd.Run(); err != nil {
        return "", fmt.Errorf("powershell failed: %s: %w",
        stderr.String(), err)
    }

    return stdout.String(), nil
}

// InstallMSI installs an MSI package silently.
```

```
func (r *Runner) InstallMSI(ctx context.Context, msiPath
        string, properties map[string]string) error {
    args := fmt.Sprintf(`/i "%s" /qn /norestart`, msiPath)
    for k, v := range properties {
        args += fmt.Sprintf(` %s="%s"`, k, v)
    }

    script := fmt.Sprintf(`Start-Process msiexec.exe
        -ArgumentList '%s' -Wait -NoNewWindow`, args)
    _, err := r.Run(ctx, script)
    return err
}

// UninstallMSI uninstalls an MSI package by product code.
func (r *Runner) UninstallMSI(ctx context.Context,
        productCode string) error {
    script := fmt.Sprintf(
        `Start-Process msiexec.exe -ArgumentList '/x "%s" /
        qn /norestart' -Wait -NoNewWindow`,
        productCode,
    )
    _, err := r.Run(ctx, script)
    return err
}

// GetPendingReboot checks if the system has a pending
        reboot.
func (r *Runner) GetPendingReboot(ctx context.Context)
        (bool, error) {
    script := `
$RebootRequired = $false

# Check Component Based Servicing
if (Test-Path "HKLM:
        \SOFTWARE\Microsoft\Windows\CurrentVersion\Component
        Based Servicing\RebootPending") {
    $RebootRequired = $true
}

# Check Windows Update
if (Test-Path "HKLM:
        \SOFTWARE\Microsoft\Windows\CurrentVersion\WindowsUpdate\Auto
        Update\RebootRequired") {
    $RebootRequired = $true
}

# Check pending file rename operations
$PendingFileRenameOps = (Get-ItemProperty "HKLM:
        \SYSTEM\CurrentControlSet\Control\Session Manager"
        -Name "PendingFileRenameOperations" -ErrorAction
        SilentlyContinue).PendingFileRenameOperations
if ($PendingFileRenameOps) {
    $RebootRequired = $true
}
```

```
Write-Output $RebootRequired
`
    output, err := r.Run(ctx, script)
    if err != nil {
        return false, fmt.Errorf("reboot check failed: %w",
        err)
    }

    return output == "True", nil
}

// GetOSVersion returns the Windows version information.
func (r *Runner) GetOSVersion(ctx context.Context) (string,
        error) {
    script := `
[System.Environment]::OSVersion.Version.ToString()
`
    output, err := r.Run(ctx, script)
    if err != nil {
        return "", fmt.Errorf("OS version query failed:
        %w", err)
    }
    return output, nil
}

// GetInstalledPrograms returns a list of installed programs
        from the registry.
func (r *Runner) GetInstalledPrograms(ctx context.Context)
        (string, error) {
    script := `
$UninstallPaths = @(
    "HKLM:
        \SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*",
    "HKLM:
        \SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*"
)

$Programs = foreach ($Path in $UninstallPaths) {
    Get-ItemProperty $Path -ErrorAction SilentlyContinue |
        Where-Object { $_.DisplayName } |
        Select-Object DisplayName, DisplayVersion,
        Publisher, InstallDate |
        ConvertTo-Json -Compress
}

$Programs
`
    return r.Run(ctx, script)
}
```

5.4 Windows Management Instrumentation (WMI)

WMI provides a standardized interface for querying and managing Windows system information. The Helix OTA agent uses WMI for hardware inventory, system health monitoring, and event subscription.

```
package wmi

import (
    "context"
    "fmt"
    "os/exec"
    "encoding/json"
)

// WMIQuery executes a WMI query and returns the results as
        JSON.
func WMIQuery(ctx context.Context, query string)
        ([]map[string]interface{}, error) {
    // Use PowerShell as a WMI wrapper for simplicity
    script := fmt.Sprintf(`
$Results = Get-CimInstance -Query "%s" | ConvertTo-Json
        -Compress
Write-Output $Results
`, query)

    cmd := exec.CommandContext(ctx,
        "powershell.exe",
        "-NoProfile", "-NonInteractive",
        "-Command", script,
    )

    output, err := cmd.CombinedOutput()
    if err != nil {
        return nil, fmt.Errorf("WMI query failed: %s: %w",
        string(output), err)
    }

    var results []map[string]interface{}
    if err := json.Unmarshal(output, &results); err != nil {
        // Single result — wrap in array
        var single map[string]interface{}
        if err2 := json.Unmarshal(output, &single); err2 !=
        nil {
            return nil, fmt.Errorf("failed to parse WMI
        output: %w", err)
        }
        results = []map[string]interface{}{single}
    }

    return results, nil
```

```
}

// SystemInfo holds WMI-derived system information.
type SystemInfo struct {
    ComputerName    string `json:"computer_name"`
    Manufacturer    string `json:"manufacturer"`
    Model           string `json:"model"`
    OSName          string `json:"os_name"`
    OSVersion       string `json:"os_version"`
    OSBuildNumber   string `json:"os_build_number"`
    ProcessorName   string `json:"processor_name"`
    TotalPhysicalMemory uint64
        `json:"total_physical_memory"`
    BIOSVersion     string `json:"bios_version"`
    SerialNumber    string `json:"serial_number"`
}

// GetSystemInfo queries WMI for comprehensive system
        information.
func GetSystemInfo(ctx context.Context) (*SystemInfo,
        error) {
    info := &SystemInfo{}

    // Operating System information
    osResults, err := WMIQuery(ctx, "SELECT Caption,
        Version, BuildNumber FROM Win32_OperatingSystem")
    if err == nil && len(osResults) > 0 {
        info.OSName = getString(osResults[0], "Caption")
        info.OSVersion = getString(osResults[0], "Version")
        info.OSBuildNumber = getString(osResults[0],
        "BuildNumber")
    }

    // Computer system information
    csResults, err := WMIQuery(ctx, "SELECT Name,
        Manufacturer, Model, TotalPhysicalMemory FROM
        Win32_ComputerSystem")
    if err == nil && len(csResults) > 0 {
        info.ComputerName = getString(csResults[0], "Name")
        info.Manufacturer = getString(csResults[0],
        "Manufacturer")
        info.Model = getString(csResults[0], "Model")
        info.TotalPhysicalMemory = getUint64(csResults[0],
        "TotalPhysicalMemory")
    }

    // Processor information
    cpuResults, err := WMIQuery(ctx, "SELECT Name FROM
        Win32_Processor")
    if err == nil && len(cpuResults) > 0 {
        info.ProcessorName = getString(cpuResults[0],
        "Name")
    }
```

```
    // BIOS information
    biosResults, err := WMIQuery(ctx, "SELECT
        SMBIOSBIOSVersion, SerialNumber FROM Win32_BIOS")
    if err == nil && len(biosResults) > 0 {
        info.BIOSVersion = getString(biosResults[0],
        "SMBIOSBIOSVersion")
        info.SerialNumber = getString(biosResults[0],
        "SerialNumber")
    }

    return info, nil
}

func getString(m map[string]interface{}, key string) string
        {
    if v, ok := m[key]; ok {
        return fmt.Sprintf("%v", v)
    }
    return ""
}

func getUint64(m map[string]interface{}, key string) uint64
        {
    if v, ok := m[key]; ok {
        switch val := v.(type) {
        case float64:
            return uint64(val)
        case string:
            var result uint64
            fmt.Sscanf(val, "%d", &result)
            return result
        }
    }
    return 0
}
```

WMI event subscription for device change monitoring:

```
# Register a WMI event subscription to detect USB device
        changes
# This can be used to trigger update checks when new
        hardware is connected
$query = "SELECT * FROM __InstanceCreationEvent WITHIN 5
        WHERE TargetInstance ISA 'Win32_LogicalDisk' AND
        TargetInstance.DriveType = 2"

Register-WmiEvent -Query $query -SourceIdentifier
        "HelixOTA_USBDeviceChange" -Action {
    $drive =
        $Event.SourceEventArgs.NewEvent.TargetInstance.DeviceID
    Write-EventLog -LogName Application -Source
        "HelixOtaAgent" `
        -EntryType Information -EventId 6001 `
```

```
        -Message "USB device connected: $drive - triggering
        update check"
}
```

---

6. New Submodules

6.1 helix-windows-client

The `helix-windows-client` submodule contains the Windows-specific OTA client implementation. It is a Go module compiled as a native Windows executable that runs as a Windows Service.

Module structure:

```
helix-windows-client/
├── go.mod                          # Module: github.com/
helix-ota/helix-windows-client
├── go.sum
├── cmd/
│   ├── helix-ota-agent/
│   │   └── main.go                 # Service executable entry
point
│   ├── helix-ota-tray/
│   │   └── main.go                 # System tray application
│   └── helix-ota-cli/
│       └── main.go                 # Command-line interface
├── internal/
│   ├── service/
│   │   ├── service.go              # Windows Service handler
│   │   ├── install.go              # Service install/
uninstall
│   │   └── recovery.go             # Service recovery
configuration
│   ├── bits/
│   │   ├── manager.go              # BITS download manager
│   │   ├── job.go                  # BITS job management
│   │   └── com.go                  # COM interop for BITS
│   ├── eventlog/
│   │   ├── logger.go               # Windows Event Log
integration
│   │   └── register.go             # Event source
registration
│   ├── ipc/
│   │   ├── pipe.go                 # Named pipe server/client
│   │   └── protocol.go             # IPC message protocol
│   ├── registry/
│   │   ├── config.go               # Registry-based
configuration
```

```
│   │   └── watch.go            # Registry change
notification
│   ├── certstore/
│   │   ├── manager.go          # Windows Certificate
Store operations
│   │   └── client.go           # mTLS client certificate
loading
│   ├── tpm/
│   │   ├── attestor.go         # TPM device attestation
│   │   └── key.go              # TPM key operations
│   ├── powershell/
│   │   └── runner.go           # PowerShell script
execution
│   ├── wmi/
│   │   ├── query.go            # WMI query execution
│   │   └── systeminfo.go       # System information
queries
│   └── tray/
│       ├── icon.go             # System tray icon
management
│       ├── notify.go           # Balloon notifications
│       └── menu.go             # Context menu
├── pkg/
│   └── adapter/
│       ├── windows.go          # WindowsAdapter
implementation
│       ├── download.go         # BITS-based download with
progress
│       ├── install.go          # MSI/MSIX installation
│       ├── verify.go           # Post-update verification
│       ├── rollback.go         # Version rollback
│       └── health.go           # Health check
implementation
├── scripts/
│   ├── install.ps1             # Installation PowerShell
script
│   ├── uninstall.ps1           # Uninstallation
PowerShell script
│   ├── health-check.ps1        # Service health check
script
│   └── configure.ps1           # Post-install
configuration
├── manifests/
│   ├── winget.yaml             # Winget package manifest
│   ├── appinstaller.xml        # MSIX app installer
config
│   └── wsus-catalog.xml        # WSUS third-party catalog
├── resources/
│   ├── helix-ota.ico           # Application icon
│   ├── app.manifest            # UAC manifest
│   └── eventlog.mc             # Event Log message file
├── test/
```

```
│   ├── integration/
│   │   ├── service_test.go       # Service lifecycle tests
│   │   ├── bits_test.go          # BITS download tests
│   │   └── msi_test.go           # MSI install/uninstall
tests
│   └── e2e/
│       ├── full_update_test.go   # End-to-end update cycle
│       └── rollback_test.go      # Rollback scenario tests
└── README.md
```

go.mod dependencies:

```
module github.com/helix-ota/helix-windows-client

go 1.23

require (
    github.com/helix-ota/core v1.2.0
    github.com/google/go-tpm v0.9.0
    github.com/go-ole/go-ole v1.3.0
    golang.org/x/sys v0.28.0
    gopkg.in/yaml.v3 v3.0.1
)
```

Key implementation milestones:

Phase	Duration	Deliverable
Phase 1: Service skeleton	2 weeks	Windows Service lifecycle, install/ uninstall, auto-start
Phase 2: BITS download	2 weeks	BITS COM integration, priority management, progress reporting
Phase 3: Event Log & IPC	1 week	Event Log integration, named pipe IPC, tray notifications
Phase 4: MSI installation	2 weeks	MSI install/uninstall via PowerShell, rollback support
Phase 5: Security	2 weeks	Certificate store, TPM attestation, Authenticode verification
Phase 6: WMI & registry	1 week	System inventory, registry configuration, health checks
Phase 7: Testing	3 weeks	Integration tests, E2E on Windows 10/11 VMs, chaos testing

6.2 helix-msi-packager

The `helix-msi-packager` submodule provides tools for creating MSI and MSIX packages. It automates the packaging, signing, and cataloging of Helix OTA agent releases.

Module structure:

```
helix-msi-packager/
├── go.mod                          # Module: github.com/
helix-ota/helix-msi-packager
├── go.sum
├── cmd/
│   └── helix-msi-packager/
│       └── main.go                 # CLI entry point
├── internal/
│   ├── msi/
│   │   ├── builder.go              # MSI package builder
│   │   ├── wix_template.go         # WiX source template
│   │   ├── transform.go            # MSI transform generation
│   │   └── validation.go           # MSI validation (ICE
checks)
│   ├── msix/
│   │   ├── builder.go              # MSIX package builder
│   │   ├── manifest.go             # AppxManifest.xml
generation
│   │   ├── appinstaller.go         # .appinstaller file
generation
│   │   └── bundle.go               # MSIX bundle creation
│   ├── signing/
│   │   ├── authenticode.go         # Authenticode signing
│   │   ├── certificate.go          # Certificate management
│   │   └── verify.go               # Signature verification
│   ├── catalog/
│   │   ├── generator.go            # Update catalog
generation
│   │   ├── delta.go                # Delta package generation
│   │   └── schema.go               # Catalog schema
definitions
│   ├── winget/
│   │   ├── manifest.go             # Winget manifest
generation
│   │   └── submit.go               # Winget-pkgs PR
submission
│   └── wsus/
│       ├── catalog.go              # WSUS catalog XML
generation
│       └── import.go               # WSUS catalog import
helper
├── templates/
│   ├── msi/
```

```
│   │   ├── agent.wxs             # WiX template for OTA
agent
│   │   └── app.wxs               # WiX template for managed
apps
│   ├── msix/
│   │   └── AppxManifest.xml      # MSIX manifest template
│   └── winget/
│       └── manifest.yaml         # Winget manifest template
├── test/
│   ├── msi_test.go               # MSI generation tests
│   ├── msix_test.go              # MSIX generation tests
│   ├── signing_test.go           # Signing verification
tests
│   └── catalog_test.go           # Catalog generation tests
└── README.md
```

CLI usage:

```
# Build an MSI package for the OTA agent
helix-msi-packager build msi \
    --binary-dir ./bin/ \
    --version 1.2.0 \
    --arch x64 \
    --output ./dist/helix-ota-agent-1.2.0-x64.msi

# Sign the MSI package
helix-msi-packager sign \
    --input ./dist/helix-ota-agent-1.2.0-x64.msi \
    --certificate ./certs/helix-ota-ev.pfx \
    --password "$CERT_PASSWORD" \
    --timestamp http://timestamp.digicert.com

# Generate the update catalog
helix-msi-packager catalog \
    --channel stable \
    --artifacts ./dist/ \
    --output ./catalogs/stable.json \
    --base-url https://ota.helix.dev/artifacts

# Generate a Winget manifest
helix-msi-packager winget \
    --package-id HelixOTA.Agent \
    --version 1.2.0 \
    --msi ./dist/helix-ota-agent-1.2.0-x64.msi \
    --output ./winget-manifests/

# Generate a WSUS catalog
helix-msi-packager wsus \
    --publisher "Helix OTA Systems" \
    --catalog-name "Helix OTA Updates" \
```

```
    --artifacts ./dist/ \
    --output ./wsus-catalog.xml
```

Delta package generation:

```
package catalog

import (
    "context"
    "fmt"
    "os/exec"
    "path/filepath"
)

// DeltaGenerator creates binary diffs between package
        versions.
type DeltaGenerator struct {
    bsdiffPath string
    zstdPath   string
}

// GenerateMSIDelta creates a bsdiff delta between two MSI
        packages.
func (d *DeltaGenerator) GenerateMSIDelta(
    ctx context.Context,
    oldMSI, newMSI, outputPatch string,
) error {
    cmd := exec.CommandContext(ctx,
        d.bsdiffPath,
        oldMSI,
        newMSI,
        outputPatch,
    )
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("bsdiff failed: %s: %w",
        string(output), err)
    }
    return nil
}

// ApplyMSIDelta applies a bsdiff delta to reconstruct the
        new MSI.
func (d *DeltaGenerator) ApplyMSIDelta(
    ctx context.Context,
    oldMSI, patchFile, outputMSI string,
) error {
    cmd := exec.CommandContext(ctx,
        filepath.Join(filepath.Dir(d.bsdiffPath),
        "bspatch"),
        oldMSI,
        outputMSI,
        patchFile,
    )
```

```
    if output, err := cmd.CombinedOutput(); err != nil {
        return fmt.Errorf("bspatch failed: %s: %w",
        string(output), err)
    }
    return nil
}
```

Implementation milestones:

Phase	Duration	Deliverable
Phase 1: MSI builder	2 weeks	WiX-based MSI generation with templates
Phase 2: Authenticode signing	1 week	signtool integration, SHA-256 signing, RFC 3161 timestamps
Phase 3: Delta generation	2 weeks	bsdiff delta generation between MSI versions
Phase 4: Catalog generation	1 week	Custom update catalog JSON, WSUS catalog XML
Phase 5: Winget integration	1 week	Winget manifest generation and submission
Phase 6: MSIX support	2 weeks	MSIX packaging for managed applications
Phase 7: CI/CD	1 week	GitHub Actions pipeline for automated packaging

---

Appendix A: Windows Version Compatibility Matrix

Windows Version	Build	Support Level	Notes
Windows 10 2004	19041	Full	Minimum supported version
Windows 10 20H2	19042	Full	
Windows 10 21H1	19043	Full	
	19044	Full	

Windows Version	Build	Support Level	Notes
Windows 10 21H2			
Windows 10 22H2	19045	Full	Last Windows 10 feature update
Windows 11 21H2	22000	Full	
Windows 11 22H2	22621	Full	
Windows 11 23H2	22631	Full	
Windows 11 24H2	26100	Full	
Windows Server 2019	17763	Full	Server Core and Desktop Experience
Windows Server 2022	20348	Full	Server Core and Desktop Experience
Windows Server 2025	26100	Full	
Windows 10 LTSC 2019	17763	Full	Long-term servicing
Windows 10 LTSC 2021	19044	Full	Long-term servicing
Windows 7 / 8 / 8.1	—	None	End of life; no support planned

Appendix B: Comparison with Android and Linux OTA Approaches

Aspect	Android (A/B)	Linux (OSTree/APT)	Windows (Helix OTA)
Update atomicity	Full partition swap	Full (OSTree) / None (APT)	MSI transaction with backup/restore

Aspect	Android (A/B)	Linux (OSTree/APT)	Windows (Helix OTA)
Rollback	Instant: boot old slot	Instant (OSTree) / Difficult (APT)	Restore backup; restart service
Download transport	Custom HTTP	HTTP / OSTree pull	BITS (idle-bandwidth-aware)
Delta support	bsdiff between images	Static deltas (OSTree) / debdelta	bsdiff between MSI packages
Package format	Full OTA ZIP	.deb / .rpm / OSTree commit	MSI / MSIX
Signing	RSA on payload	GPG on repository metadata	Authenticode (EV certificate)
Service architecture	update_engine daemon	systemd service / cron	Windows Service (SCM-managed)
User notification	System notification	Desktop notification	System tray balloon + Event Log
Enterprise management	EMM / MDM	Landscape / Spacewalk	Intune / MECM / GPO
Distribution channel	OTA server	APT/RPM repository	Winget / MSI direct / Intune
Configuration store	/system/build.prop	/etc configuration files	Windows Registry
Telemetry	Checkin protocol	Custom reporting	Event Log + Helix telemetry API

Appendix C: Security Checklist

- [ ] All executables signed with EV Authenticode certificate
- [ ] MSI packages include embedded digital signatures

- [ ] Client certificate stored in Windows Certificate Store (not flat file)
- [ ] Private keys protected by DPAPI or TPM
- [ ] BITS downloads verify SHA-256 hashes before installation
- [ ] Update catalog served over HTTPS with certificate pinning
- [ ] Service runs as dedicated account with minimum required privileges
- [ ] Named pipe IPC uses appropriate ACLs (restrict to LocalSystem and Administrators)
- [ ] Event Log source registered during installation
- [ ] Registry keys secured with appropriate permissions
- [ ] UAC manifest declares `asInvoker` for user-facing binaries
- [ ] Rollback backup directory ACL’d to LocalSystem only
- [ ] SmartScreen reputation established before consumer deployment
- [ ] TPM attestation for device identity (where TPM is available)
- [ ] Firewall rules configured for outbound HTTPS to OTA server only